Έγινε λήψη της απάντησης ης εφαρμογής

Έγινε λήψη της απάντησης ης εφαρμογής

Σκέφτηκε για 6m 52s

Final repository inspection result

Το repository `ntua-el21026/NTUA-saasPlug` φαίνεται **πλήρες και επαρκές για τελική υποβολή μαθήματος**, με βάση τα πραγματικά αρχεία του repo, όχι μόνο το prompt. Η υλοποίηση ακολουθεί τα canonical rules του `AGENTS.md` και το accepted baseline του `docs/specs/SPECIFICATIONS.md`: React/Vite frontend, FastAPI microservices, Nginx gateway, RabbitMQ μόνο για async/background flows, ένα PostgreSQL container ανά stateful service, εσωτερικά provider services, final docs/UML/AI logs, και τελικό verification report.

Το `COMPLETED_WITH_WARNINGS` του Prompt 14 είναι **αποδεκτό**: τα warnings είναι τεκμηριωμένα tooling/environment limitations, όχι αρχιτεκτονικά ή λειτουργικά blockers. Το final verification report καταγράφει passing frontend tests/build, shared event tests με `-s` workaround, Docker Compose startup/health, gateway health, authenticated EV/provider/operator smoke checks, no `.env`, και forbidden frontend pattern checks.

final-verification

What is confirmed

Area	Status	Evidence / notes
Course/project identity	Confirmed	`AGENTS.md` ορίζει το project ως NTUA ECE Technologies of Software Services SaaS web app, με repo content στα English και discussion στα Greek. AGENTS
Canonical architecture	Confirmed	Accepted stack: React + TypeScript + Vite, FastAPI, Nginx gateway, RabbitMQ topic exchange, one PostgreSQL container per stateful service, PlantUML diagrams. AGENTS
Specification alignment	Confirmed	`docs/specs/SPECIFICATIONS.md` είναι accepted project baseline και explicitly prevents architecture drift. SPECIFICATIONS
Service set	Confirmed	Τα canonical services είναι ακριβώς παρόντα στο spec και services README: `auth-service`, `provider-registry-service`, `red/green/blue-provider-service`, `catalog-service`, `status-service`, `reservation-service`, `analytics-service`, `billing-service`. SPECIFICATIONS README
Service responsibilities	Confirmed	Services README αντιστοιχίζει κάθε service σε bounded responsibility και owned database. README
One PostgreSQL per stateful service	Confirmed	`docker-compose.yml` περιέχει ξεχωριστά `auth-postgres`, `provider-registry-postgres`, `red-provider-postgres`, `green-provider-postgres`, `blue-provider-postgres`, `catalog-postgres`, `status-postgres`, `reservation-postgres`, `analytics-postgres`, `billing-postgres`. docker-compose
No MongoDB / no shared DB	Confirmed	Spec απαγορεύει MongoDB για MVP και cross-service SQL joins/writes/shared mutable tables. SPECIFICATIONS
Gateway-only frontend	Confirmed	Frontend API helper rejects any path not starting with `/api/`; fetch uses gateway-relative path. api
Provider services internal	Confirmed	Nginx exposes auth/providers/catalog/status/reservations/analytics/billing only; no red/green/blue provider upstreams. default default

Area	Status	Evidence / notes
Provider API mapping	Confirmed	Provider configs map actual red/green/blue YAML routes and fields to internal adapters. provider_config provider_config provider_config
Catalog bounds loading	Confirmed	`catalog-service` implements `minLat`, `maxLat`, `minLng`, `maxLng` bounds filtering and rejects incomplete/invalid bounds. routes
Status cache	Confirmed	`status-service` implements allowed status normalization, cache hit behavior, 5-minute TTL via settings, provider live lookup, and fallback to last known cached status. status_logic status_logic
Reservation flow	Confirmed	Reservation is synchronous: resolve catalog point, call provider service, then store success/failure/unknown and create outbox event. reservation_logic
RabbitMQ boundaries	Confirmed	`packages/events` defines one exchange `saasplug.events`, canonical routing keys, queues/DLQs, and idempotency rules; docs state RabbitMQ is not used for login/map/details/status/reservation provider calls/live frontend actions. README README architecture-summary
Frontend routes	Confirmed	`App.tsx` includes EV, provider-admin, and operator route groups with role guards. App App App
Final docs	Confirmed	Root README lists architecture, runtime, data ownership, demo users, core flows, validation, and submission artifacts. README README
Final PlantUML diagrams	Confirmed	`docs/diagrams/final/README.md` lists all 13 required `.puml` files. README
Deployment diagram coherence	Confirmed	Deployment diagram includes Docker Compose, gateway host `8080 -> 80`, frontend internal `5173`, FastAPI internal `8000`, RabbitMQ ports, and all PostgreSQL internal `5432` containers. deployment
Data model diagram coherence	Confirmed	Data model diagram separates persistence by service/database and notes cross-package links are logical

Area	Status	Evidence / notes
		IDs/events/projections, not cross-service foreign keys. data-model data-model
AI logs	Confirmed	Prompt log indexes runs `000` through `014` with the expected completed / completed_with_warnings statuses. prompt-log
GitHub issues #1-#14	Confirmed closed	Closed issue search returns issues #1 through #14. Prompt 01 - Repository foundati… Issue #14 is closed with `state_reason: completed`. Prompt 14 - Final docs and UML …
GitHub Project state	Partially confirmed	Prompt 14 run report says Codex verified issue #14 project item as `Done`; my connector did not expose a direct GitHub Projects read API, so I can confirm this through the committed run report, not independently through a project-board connector. 014-final-docs-uml
Fallback project-management Markdown	No evidence found	`AGENTS.md` forbids fallback Markdown project-management artifacts unless explicitly requested, and Prompt 14 reports none were created. AGENTS 014-final-docs-uml
Final verification	Confirmed	Final verification says repo is submission-ready subject to documented local MVP limitations. final-verification

Warnings / non-blocking issues

1. **Google OAuth is hook-level for the local MVP.** Email/password/JWT login is implemented, but real Google token exchange is intentionally deferred. This is explicitly documented as an MVP limitation.

   routes

   architecture-summary

2. **Live provider calls depend on real provider credentials.** `.env.example` contains placeholders, and docs state live provider calls may fail until real credentials are placed in an uncommitted `.env`.

   .env

   architecture-summary

3. **No scheduler/worker container exists.** The MVP uses manual operator endpoints for imports, event drains, aggregate refresh, and outbox publishing. This is documented, and it does not conflict with the accepted MVP.

   architecture-summary

4. **Startup table creation is used instead of full Alembic migration management.** This is a documented MVP limitation.

architecture-summary

5. **PlantUML source is present, but local rendering was not validated.** The `plantuml` CLI was not installed in the local WSL environment during Prompt 14.

final-verification

6. **Browser visual automation and Figma validation were not run.** Browser Use localhost automation was unavailable, and no usable Figma source was provided.

final-verification

7. **Shared event tests required a local pytest workaround.** `pytest -q` hit a Python 3.14 capture issue; `pytest -q -s` passed all 10 shared event tests.

final-verification

Blockers

No blockers.

The evidence supports final submission readiness: Prompt 14's final verification passed frontend tests/build, shared event tests with workaround, Docker Compose config/startup/health, gateway health, authenticated EV/provider/operator smoke checks, static forbidden-pattern checks, final file existence checks, and Docker cleanup.

final-verification

Prompt 14 explicitly records that no blockers remain for the final documentation scope.

014-final-docs-uml

Final Codex full verification prompt

```
You are working in the repository:

ntua-el21026/NTUA-saasPlug

Task type:

FINAL VERIFICATION ONLY.

Do not implement new features.
Do not change backend or frontend business logic.
Do not change architecture.
Do not add services, databases, brokers, workers, schedulers, orchestrators, deploy
Do not create `.env`.
Do not commit or print secrets.
```

```
Do not close, reopen, mutate, label, or comment on GitHub issues unless explicitly
Do not create fallback Markdown project-management artifacts such as `docs/project-
Repository files, comments, docs, prompts, and logs must remain in English.

Canonical sources:

1. Read `AGENTS.md`.
2. Read `docs/specs/SPECIFICATIONS.md`.
3. Read `README.md`.
4. Read `docs/architecture-summary.md`.
5. Read `docs/final-verification.md`.
6. Read `docs/submission-checklist.md`.
7. Read `docs/ai/prompt-log.md`.
8. Read `docs/ai/codex-runs/014-final-docs-uml.md`.

Verification goals:

Confirm that the repository is still ready for final course submission and still ma

- one role-based React + TypeScript + Vite frontend under `apps/frontend`;
- browser calls only gateway-relative `/api/...` paths;
- no frontend calls to internal service ports, provider-service hostnames, external
- Python FastAPI backend services;
- canonical service set only:
  - `auth-service`
  - `provider-registry-service`
  - `red-provider-service`
  - `green-provider-service`
  - `blue-provider-service`
  - `catalog-service`
  - `status-service`
  - `reservation-service`
  - `analytics-service`
  - `billing-service`
- provider services remain internal and are not exposed directly through Nginx;
- RabbitMQ is used only for background/event flows, not login, map search, point de
- one PostgreSQL container per stateful service;
- no MongoDB;
- no dedicated orchestrator service;
- final documentation and PlantUML source diagrams are present and coherent;
- AI logs are complete and consistent.

Run these checks where available.

General repository checks:

```bash
git status --short
git diff --check
test ! -f .env
docker compose config
```
```

Frontend checks:

```bash
cd apps/frontend
npm run test
npm run build
cd ../..
```

Shared event package checks:

```bash
PYTHONPATH=packages/events python -m pytest packages/events/tests -q
```

If the command above fails only because of a local Python/pytest capture issue, retry exactly:

```bash
PYTHONPATH=packages/events python -m pytest packages/events/tests -q -s
```

Clearly report both the original failure and the retry result. Do not hide the workaround.

Backend/service test discovery:

- Inspect service-local test directories under `services/*/tests`.
- Run available service tests if dependencies and Docker/local Python environment make them runnable.
- If a test suite cannot be run because the required local environment or dependency installation is unavailable, report the exact limitation instead of claiming success.

Docker runtime checks, only if Docker is available:

```bash
docker info
docker compose up -d --build
docker compose ps
```

Then verify gateway and public service health through the gateway:

```bash
curl -fsS http://localhost:8080/health
curl -fsS http://localhost:8080/
curl -fsS http://localhost:8080/api/auth/health
curl -fsS http://localhost:8080/api/providers/health
```

```
curl -fsS http://localhost:8080/api/catalog/health
curl -fsS http://localhost:8080/api/status/health
curl -fsS http://localhost:8080/api/reservations/health
curl -fsS http://localhost:8080/api/analytics/health
curl -fsS http://localhost:8080/api/billing/health
```

Run authenticated smoke checks through the gateway:

1. EV user:
   - login as `evuser@saasplug.local` using the safe demo password from `.env.example`;
   - call `/api/auth/auth/me`;
   - verify the returned role includes `EV_USER`.
2. Provider admin:
   - login as `red-admin@saasplug.local`;
   - call a provider-scoped dashboard endpoint through the gateway, for example provider analytics summary if available;
   - verify it succeeds and remains provider-scoped.
3. Operator:
   - login as `operator@saasplug.local`;
   - call an operator dashboard endpoint through the gateway, for example operator analytics summary if available;
   - verify it succeeds.

Do not print access tokens or refresh tokens in the final response or logs.

After Docker runtime checks, clean up:

Bash

```
docker compose down -v
docker compose ps -a
```

Static frontend forbidden-call check:

Run a robust `rg` check over `apps/frontend/src` for forbidden direct calls or leaked infrastructure references. At minimum, check for:

- `localhost`
- `127.0.0.1`
- `:8000`
- `:8001`
- `:8002`
- `:8011`

- `:8012`
- `:8013`
- `:8020`
- `:8021`
- `:8030`
- `:8040`
- `:8050`
- `red-provider-service`
- `green-provider-service`
- `blue-provider-service`
- `davinci.softlab.ntua.gr`
- `amqp://`
- `rabbitmq`
- `/api/red-provider`
- `/api/green-provider`
- `/api/blue-provider`

Use `rg -e` arguments instead of shell pipes if quoting is fragile.

Final documentation and diagram presence check:

Verify these files exist:

```
AGENTS.md
README.md
.env.example
.gitignore
docker-compose.yml
deploy/nginx/default.conf
docs/specs/SPECIFICATIONS.md
docs/specs/README.md
docs/architecture-summary.md
docs/final-verification.md
docs/submission-checklist.md
docs/README.md
docs/ai/prompt-log.md
docs/ai/codex-runs/014-final-docs-uml.md
docs/diagrams/final/README.md
docs/diagrams/final/use-case.puml
docs/diagrams/final/component.puml
docs/diagrams/final/deployment.puml
docs/diagrams/final/data-model.puml
docs/diagrams/final/activity-search-reservation.puml
docs/diagrams/final/activity-provider-sync.puml
docs/diagrams/final/activity-invoice-payment.puml
docs/diagrams/final/sequence-login.puml
```

```
docs/diagrams/final/sequence-search-map.puml
docs/diagrams/final/sequence-status-lookup.puml
docs/diagrams/final/sequence-reservation.puml
docs/diagrams/final/sequence-provider-sync-catalog-import.puml
docs/diagrams/final/sequence-analytics-billing-events.puml
docs/assignment/providers/yaml/redPlug.yaml
docs/assignment/providers/yaml/greenPlug.yaml
docs/assignment/providers/yaml/bluePlug.yaml
```

PlantUML validation:

- If `plantuml` is installed, validate/render the final `.puml` files.
- If `plantuml` is not installed, clearly report that PlantUML source files were not locally rendered because the CLI is unavailable.

AI logs and issue/project checks:

- Verify `docs/ai/prompt-log.md` contains runs `000`, `00B`, and `001` through `014`.
- Verify each listed run report exists under `docs/ai/codex-runs/`.
- If GitHub CLI/API access is available, read-only inspect issues #1 through #14 and report whether they are closed/completed.
- If GitHub Project state is not visible, say so clearly.
- Do not close, reopen, comment on, relabel, or otherwise mutate any GitHub issue or Project item.

Reporting and allowed file updates:

Create a new verification-only run report under:

```
docs/ai/codex-runs/015-final-verification-only.md
```

The report must include:

- exact invocation prompt summary;
- canonical sources read;
- commands run;
- pass/fail/skipped/unavailable result for each check;
- Docker availability and runtime results;
- authenticated smoke results without tokens;
- PlantUML availability/render result;
- frontend forbidden-pattern result;
- final docs/diagrams presence result;
- issue/project read-only result if available;
- warnings;
- blockers;
- final conclusion.

Update `docs/ai/prompt-log.md` only if you create the new run report entry.

Update `docs/final-verification.md` only if the actual verification results differ from the current report. If nothing materially changes, leave it unchanged and state that in the run report.

Do not modify application source, backend source, frontend source, Docker topology, Nginx routes, service schemas, provider adapters, RabbitMQ contracts, business logic, specs, assignment sources, or lecture sources.

Final response requirements:

Return:

1. Overall status: `PASS`, `PASS_WITH_WARNINGS`, or `FAIL`.
2. Commands run and results.
3. Files changed.
4. Warnings.
5. Blockers.
6. Whether the repository is still ready for final course submission.

Be precise. If a tool, dependency, Docker engine, Browser Use, Figma, PlantUML, GitHub issue access, or GitHub Project access is unavailable, report it explicitly instead of claiming success.



Now, we need to do 2 things, bearing in mind that the chat with Codex will be a new one, so we also need to provide it with the relevant context precisely:
1. Close the current project (see screenshot) and also create a new one (with proper issues in there), where we will: a. audit the repo for all implementation points (1-14) b. connect real environmental variables and check real functionality c. resolve all the warnings 1-7 that appeared (we need a scheduler, not only manual workflows) d. add proper tests throughout the whole app (in a tests/ dir under root ideally). a,b,c,d in this order

Εμφάνιση περισσότερων

Σκέφτηκε για 33s

Ναι. Το πρώτο prompt πρέπει να κάνει **μόνο project closure + new project planning + Phase A audit**, όχι να αρχίσει fixes για env vars, scheduler ή tests.

Παρακάτω είναι έτοιμο copy-paste prompt για νέο Codex chat.

```
You are working in the GitHub repository:

ntua-el21026/NTUA-saasPlug
```

Project name:

saasPlug

Course context:

This is an NTUA ECE semester project for the course “Technologies of Software Servi

Repository content rule:

All repository files, documentation, code, comments, README files, Codex prompts, A

Do not use Greek in repository artifacts or GitHub project-management artifacts.

Do not use Vercel.
Do not add deployment or hosting workflows.
Do not implement application features in this prompt.
Do not change backend business logic.
Do not change frontend business logic.
Do not change architecture.
Do not add secrets.
Do not create `.env`.
Do not print secret values.
Do not create fallback Markdown project-management artifacts such as `docs/project-
Use GitHub Projects/issues directly if the available GitHub CLI/API permissions all
If GitHub Project operations are unavailable, create/read issues if possible, and c

Canonical repository sources:

Treat these as authoritative:

- AGENTS.md
- README.md
- docs/specs/SPECIFICATIONS.md
- docs/ai/prompt-log.md
- docs/ai/codex-runs/
- docs/ai/prompts/README.md
- docs/final-verification.md
- docs/submission-checklist.md
- docs/architecture-summary.md
- docs/diagrams/final/

If anything conflicts with `AGENTS.md` or `docs/specs/SPECIFICATIONS.md`, follow `A

Important current state:

The first implementation project was completed through staged prompts/issues #1 thr

Expected historical issue/run mapping:

- Issue #1 / Prompt 01 - Repository foundation
- Issue #2 / Prompt 02 - Docker Compose and Nginx gateway

- Issue #3 / Prompt 03 - Auth service
- Issue #4 / Prompt 04 - Provider registry service
- Issue #5 / Prompt 05 - red/green/blue provider services
- Issue #6 / Prompt 06 - Catalog service
- Issue #7 / Prompt 07 - Status service
- Issue #8 / Prompt 08 - Reservation service
- Issue #9 / Prompt 09 - RabbitMQ event infrastructure
- Issue #10 / Prompt 10 - Analytics service
- Issue #11 / Prompt 11 - Billing service
- Issue #12 / Prompt 12 - EV frontend flows
- Issue #13 / Prompt 13 - Provider and operator dashboards
- Issue #14 / Prompt 14 - Final docs and UML finalization

Expected run statuses in `docs/ai/prompt-log.md`:

- 000: completed
- 00B: completed_with_warnings
- 001: completed
- 002: completed
- 003: completed
- 004: completed
- 005: completed
- 006: completed_with_warnings
- 007: completed_with_warnings
- 008: completed
- 009: completed
- 010: completed
- 011: completed
- 012: completed_with_warnings
- 013: completed_with_warnings
- 014: completed_with_warnings

Prompt 14 final verification previously passed:

- frontend tests;
- frontend build;
- shared events tests with `-s` workaround;
- `docker compose config`;
- `docker compose up -d --build`;
- `docker compose ps` with healthy containers;
- gateway health;
- public service health routes;
- authenticated EV smoke;
- authenticated provider-admin smoke;
- authenticated operator smoke;
- Docker cleanup;
- `git diff --check`;
- no `.env`;
- forbidden frontend/source checks;
- README/final docs/diagram presence checks.

Known non-blocking warnings from final inspection:

````
1. Google OAuth is hook-level/local-MVP only; real token exchange is deferred.
2. Live provider calls require real provider credentials/environment variables.
3. No scheduler/worker container exists; workflows are manually triggered by operat
4. Startup table creation is used instead of a complete migration workflow.
5. PlantUML source exists, but local rendering was previously not validated becaus
6. Browser visual automation / design QA and Figma validation were not previously r
7. Shared event tests previously required a Python/pytest `-s` workaround due to a

Important environment update:

PlantUML verification tools have now been installed locally with:

```bash
sudo apt install default-jre graphviz plantuml
````

Therefore this audit should attempt actual PlantUML validation/rendering.

Accepted architecture summary:

1. One role-based frontend web app:
   - React + TypeScript + Vite.
   - Located under `apps/frontend`.
   - Browser calls only gateway routes under `/api/...`.
   - No browser calls to internal service ports.
   - No browser calls to external provider APIs.
   - No browser RabbitMQ access.
2. Backend:
   - Python FastAPI microservices.
   - Each service owns its data and its own PostgreSQL container if stateful.
3. Gateway:
   - Nginx reverse proxy/API gateway.
   - Frontend calls only gateway paths.
   - No direct browser access to internal services or provider APIs.
4. Runtime:
   - Docker Compose.
5. Databases:
   - PostgreSQL only.
   - Separate PostgreSQL container per stateful service.
   - No MongoDB.
6. Messaging:
   - RabbitMQ topic exchange.
   - RabbitMQ is used for background/event flows:
     - provider sync;
````

        - catalog aggregation/import events;
        - analytics aggregation;
        - billing propagation.
    - RabbitMQ is not used for:
        - login;
        - map search;
        - point details;
        - status lookup;
        - the live reservation provider call;
        - direct frontend actions.
7. No dedicated orchestrator service:
    - Use local orchestration inside responsible services.
    - Use event choreography for asynchronous propagation.
    - No saga/distributed transaction implementation for MVP.

Canonical service set:

- auth-service
- provider-registry-service
- red-provider-service
- green-provider-service
- blue-provider-service
- catalog-service
- status-service
- reservation-service
- analytics-service
- billing-service

Your task in this prompt:

This is Phase A of the new finalization process.

Perform exactly these high-level actions:

1. Close the completed current GitHub Project if possible.
2. Create a new GitHub Project for finalization/hardening if possible.
3. Create proper GitHub issues for the new project, ordered by the planned sequence:
    - A. Audit the repo for all implementation points 1-14.
    - B. Connect real environment variables and verify real functionality.
    - C. Resolve documented warnings 1-7, including adding scheduler/worker support.
    - D. Add proper tests throughout the app, ideally under a root-level `tests/` directory.

4. Execute Phase A only: audit the repo for all implementation points 1-14 and rerun verification checks.
5. Document this prompt and run in the existing AI log system.

Do not start Phase B, Phase C, or Phase D implementation in this prompt.

GitHub Project operations:

First inspect available GitHub/gh permissions:

```bash
gh auth status
gh repo view ntua-el21026/NTUA-saasPlug
gh project list --owner ntua-el21026
```

Current project expected from browser UI:

- Existing open project: `saasPlug Implementation`
- Project number likely: `1`

If accessible:

1. Inspect current project #1:

```bash
gh project view 1 --owner ntua-el21026
gh project item-list 1 --owner ntua-el21026 --limit 100
```

2. Confirm that the implementation project is complete enough to close:
   - Issues #1 through #14 are closed/completed.
   - Prompt log contains runs through 014.
   - `docs/ai/codex-runs/014-final-docs-uml.md` exists.
3. If confirmed, close project #1:

```bash
gh project close 1 --owner ntua-el21026
```

If project close is not available through the installed `gh` version or permissions, do not attempt destructive alternatives. Report the exact limitation.

Create the new GitHub Project if possible:

Suggested project title:

```
saasPlug Finalization and Real-World Verification
```

Suggested command:

```bash
gh project create --owner ntua-el21026 --title "saasPlug Finalization and Real-World Veri
```

After creation, capture the new project number from `gh project list --owner ntua-el21026`.

Create these GitHub issues in `ntua-el21026/NTUA-saasPlug` and add them to the new project if project item operations are available.

Issue 1:

Title:

```
Phase A - Audit implementation points 001-014
```

Body:

```markdown
## Purpose

Perform a full verification-only audit of the existing saasPlug implementation across pro

## Scope

- Inspect canonical repository governance and specifications.
- Verify issues #1-#14 and AI run reports.
- Verify all canonical services and responsibility boundaries.
- Verify frontend gateway-only API usage.
- Verify provider services remain internal.
- Verify RabbitMQ usage remains restricted to background/event flows.
- Verify one PostgreSQL container per stateful service.
- Verify final docs and PlantUML source diagrams.
- Rerun tests/builds/static checks.
- Rerun Docker Compose and authenticated smoke checks if Docker is available.
- Validate PlantUML rendering now that `default-jre`, `graphviz`, and `plantuml` are inst

## Out of scope

- No feature implementation.
- No business logic changes.
- No architecture changes.
- No secrets or `.env` creation.
```

```
- No deployment.

## Acceptance criteria

- A new AI run report is created under `docs/ai/codex-runs/`.
- `docs/ai/prompt-log.md` is updated with the new run.
- The audit clearly reports PASS / PASS_WITH_WARNINGS / FAIL.
- Any blockers are listed precisely.
- Any non-blocking warnings are listed precisely.
- The repository remains unchanged except allowed AI-log/final-verification documentation
```

Issue 2:

Title:

```
Phase B - Configure real environment variables and verify live functionality
```

Body:

Markdown

```
## Purpose

Connect real local environment variables and verify real saasPlug runtime functionality a

## Scope

- Prepare `.env` locally only; never commit it.
- Validate `.env.example` completeness.
- Configure provider API credentials and runtime settings locally.
- Verify live provider sync/status/reservation behavior where credentials allow.
- Verify Google OAuth behavior if real credentials are available.
- Verify end-to-end user, provider-admin, and operator flows through the gateway.

## Out of scope

- No secrets committed.
- No deployment.
- No architecture replacement.

## Acceptance criteria

- `.env` remains untracked.
- Real functionality checks are documented without printing secrets.
- Failures distinguish between code issues, credential issues, and provider endpoint issu
```

Issue 3:

Title:

```
Phase C1 - Complete Google OAuth real token exchange
```

Body:

Markdown

```
## Purpose

Resolve warning 1: Google OAuth is currently hook-level/local-MVP only.

## Scope

- Implement or finalize real Google OAuth callback/token exchange only if credentials are
- Preserve email/password login.
- Preserve JWT/refresh-token model.
- Preserve role model: `EV_USER`, `PROVIDER_ADMIN`, `OPERATOR`.

## Acceptance criteria

- Google OAuth works with real local credentials.
- Placeholder/local mode remains safe.
- Tests cover configured and unconfigured OAuth modes.
```

Issue 4:

Title:

```
Phase C2 - Verify live provider credentials and provider endpoint behavior
```

Body:

Markdown

```
## Purpose

Resolve warning 2: live provider calls require real provider credentials/environment vari

## Scope

- Verify redPlug, greenPlug, and bluePlug live endpoint behavior with real local credenti
- Confirm provider adapters match assignment OpenAPI YAML files.
- Document provider endpoint failures precisely.

## Acceptance criteria

- Real sync/status/reservation checks are documented.
- Secrets are never printed or committed.
- Provider-specific failures are classified.
```

Issue 5:

Title:

```
Phase C3 - Add scheduler/worker support for background workflows
```

Body:

Markdown

```
## Purpose

Resolve warning 3: workflows are currently manually triggered by operator endpoints only.

## Scope

Add scheduler/worker support for background workflows while preserving the accepted micro

Required scheduled/background workflows:

- provider sync;
- catalog import/aggregation;
- analytics aggregation refresh;
- billing event drain/propagation;
- reservation outbox publishing if applicable.

## Constraints

- Do not add a centralized business orchestrator.
- Do not use RabbitMQ for live frontend actions.
- Do not route login/search/status/reservation live calls through queues.
- Preserve event choreography for asynchronous propagation.
- Use Docker Compose-compatible local runtime.

## Acceptance criteria

- Scheduler/worker is documented.
- Scheduled jobs are configurable.
- Manual operator controls continue to work.
- Tests cover scheduled job invocation and safe failure handling.
```

Issue 6:

Title:

```
Phase C4 - Replace startup table creation with proper migration workflow
```

Body:

Markdown

```
## Purpose

Resolve warning 4: startup table creation is used instead of a complete migration workflo

## Scope

- Introduce or finalize Alembic-style migrations for stateful FastAPI services.
- Preserve separate database ownership per service.
- Avoid cross-service database joins/writes.
- Keep Docker Compose startup deterministic.

## Acceptance criteria

- Each stateful service has a reproducible migration path.
- Fresh Docker Compose startup works.
- Existing demo seed data still works.
- Migration commands are documented.
```

Issue 7:

Title:

```
Phase C5 - Validate and render final PlantUML diagrams
```

Body:

Markdown

```
## Purpose

Resolve warning 5: PlantUML source existed but rendering was not previously validated.

## Scope

- Validate all final `.puml` files under `docs/diagrams/final/`.
- Render diagrams locally using installed `default-jre`, `graphviz`, and `plantuml`.
- Fix only diagram syntax/documentation issues if needed.

## Acceptance criteria

- All final PlantUML diagrams render successfully.
- Render output policy is documented.
- Source `.puml` files remain authoritative.
```

Issue 8:

Title:

```
Phase C6 - Add browser/UI automation and design QA verification
```

Body:

Markdown

```
## Purpose

Resolve warning 6: browser visual automation / design QA and Figma validation were not pr

## Scope

- Add feasible browser/UI automation for the local app.
- Validate core EV, provider-admin, and operator screens.
- If no Figma source is available, document that explicitly.
- Do not require deployment.

## Acceptance criteria

- Browser/UI checks run locally against gateway.
- Core routes are covered.
- Any unavailable Figma source is reported clearly.
```

Issue 9:

Title:

```
Phase C7 - Resolve Python/pytest compatibility warning
```

Body:

Markdown

```
## Purpose

Resolve warning 7: shared event tests previously required a `-s` workaround due to a loca

## Scope

- Identify whether the issue is local Python 3.14-specific, pytest-specific, or repositor
- Prefer deterministic test commands that work without special capture workarounds.
- Do not hide legitimate test failures.

## Acceptance criteria

- Shared event tests run reliably with documented commands.
- The workaround is removed if possible or explicitly documented if environment-specific.
```

Issue 10:

Title:

```
Phase D - Add root-level application test suite
```

Body:

Markdown

```
## Purpose

Add a proper root-level test strategy for the whole app.

## Scope

- Add a root-level `tests/` directory if appropriate.
- Cover cross-service integration flows.
- Cover gateway route behavior.
- Cover authenticated EV, provider-admin, and operator flows.
- Cover event propagation assumptions.
- Cover scheduler workflows after Phase C3.
- Keep service-local unit tests intact.

## Constraints

- Do not duplicate all service-local unit tests unnecessarily.
- Do not commit secrets.
- Do not require deployment.
- Tests must be runnable locally.

## Acceptance criteria

- Root-level tests are documented.
- Tests can run against Docker Compose where required.
- CI/local command documentation is clear.
- Existing frontend and service tests continue to pass.
```

Add the issues to the new project in the exact order above if possible. If the GitHub Project supports fields/statuses, set initial statuses as:

- Phase A: `In Progress`
- Phase B: `Backlog`
- Phase C1-C7: `Backlog`
- Phase D: `Backlog`

If status fields are not configurable through available tools, add the items and report that field setup was unavailable.

Phase A repository audit:

After project/issue setup, perform the full audit requested below.

Do not rely only on previous reports. Inspect the repository directly.

Inspect at least:

Root and governance:

- AGENTS.md
- README.md
- .gitignore
- .env.example
- docker-compose.yml
- deploy/nginx/default.conf

Specs and final docs:

- docs/specs/SPECIFICATIONS.md
- docs/specs/README.md
- docs/architecture-summary.md
- docs/final-verification.md
- docs/submission-checklist.md
- docs/README.md

AI logs:

- docs/ai/prompt-log.md
- docs/ai/codex-runs/000-audit-only.md
- docs/ai/codex-runs/00B-logging-and-github-setup.md
- docs/ai/codex-runs/001-repo-foundation.md
- docs/ai/codex-runs/002-docker-gateway.md
- docs/ai/codex-runs/003-auth-service.md
- docs/ai/codex-runs/004-provider-registry-service.md
- docs/ai/codex-runs/005-provider-services.md
- docs/ai/codex-runs/006-catalog-service.md
- docs/ai/codex-runs/007-status-service.md
- docs/ai/codex-runs/008-reservation-service.md
- docs/ai/codex-runs/009-rabbitmq-event-infrastructure.md
- docs/ai/codex-runs/010-analytics-service.md
- docs/ai/codex-runs/011-billing-service.md
- docs/ai/codex-runs/012-frontend-ev-flows.md
- docs/ai/codex-runs/013-provider-operator-dashboards.md
- docs/ai/codex-runs/014-final-docs-uml.md

Frontend:

- apps/frontend/README.md
- apps/frontend/package.json
- apps/frontend/package-lock.json
- apps/frontend/src/App.tsx
- apps/frontend/src/components/
- apps/frontend/src/features/
- apps/frontend/src/lib/
- apps/frontend/src/styles/
- apps/frontend/src/test/

Shared package:

- packages/events/README.md
- packages/events/saasplug_events/
- packages/events/tests/

Services:

- services/README.md
- services/auth-service/
- services/provider-registry-service/
- services/red-provider-service/
- services/green-provider-service/
- services/blue-provider-service/
- services/catalog-service/
- services/status-service/
- services/reservation-service/
- services/analytics-service/
- services/billing-service/

Final diagrams:

- docs/diagrams/final/README.md
- all `.puml` files under `docs/diagrams/final/`

Assignment/provider sources:

- docs/assignment/
- docs/assignment/providers/yaml/redPlug.yaml
- docs/assignment/providers/yaml/greenPlug.yaml
- docs/assignment/providers/yaml/bluePlug.yaml

GitHub:

- Inspect issues #1 through #14.
- Confirm whether issues are closed/completed.
- Inspect issue #14 especially.

- Confirm whether there are no open historical implementation issues.
- Confirm whether there are no fallback Markdown project-management artifacts such as `docs/project-management`.
- Confirm the old and new GitHub Project state if visible.
- If GitHub Project state is not visible, say that clearly.

Run verification commands.

General checks:

```bash
git status --short
git diff --check
test ! -f .env
docker compose config
```

Frontend checks:

```bash
cd apps/frontend
npm run test
npm run build
cd ../..
```

Shared event checks:

```bash
PYTHONPATH=packages/events python -m pytest packages/events/tests -q
```

If and only if the command above fails because of the known local Python/pytest capture issue, retry:

```bash
PYTHONPATH=packages/events python -m pytest packages/events/tests -q -s
```

Report both results.

PlantUML checks:

```bash
java -version
dot -V
plantuml -version
```

```
plantuml -checkonly docs/diagrams/final/*.puml
plantuml -tpng docs/diagrams/final/*.puml
```

If `-checkonly` is unsupported in the installed PlantUML version, use a rendering-only validation and report the version limitation.

Docker runtime checks, if Docker is available:

Bash

```
docker info
docker compose up -d --build
docker compose ps
```

Then verify gateway/public health routes:

Bash

```
curl -fsS http://localhost:8080/health
curl -fsS http://localhost:8080/
curl -fsS http://localhost:8080/api/auth/health
curl -fsS http://localhost:8080/api/providers/health
curl -fsS http://localhost:8080/api/catalog/health
curl -fsS http://localhost:8080/api/status/health
curl -fsS http://localhost:8080/api/reservations/health
curl -fsS http://localhost:8080/api/analytics/health
curl -fsS http://localhost:8080/api/billing/health
```

Run authenticated smoke checks through the gateway:

1. EV user:
   - login as `evuser@saasplug.local` using the safe demo password from `.env.example`;
   - call `/api/auth/auth/me`;
   - verify returned role includes `EV_USER`.
2. Provider admin:
   - login as `red-admin@saasplug.local`;
   - call a provider-admin reachable endpoint through the gateway;
   - verify it succeeds and remains provider-scoped.
3. Operator:
   - login as `operator@saasplug.local`;
   - call an operator reachable endpoint through the gateway;
   - verify it succeeds.

Do not print access tokens or refresh tokens in logs or final response.

After Docker runtime checks:

```bash
docker compose down -v
docker compose ps -a
```

Static frontend forbidden-call check:

Run robust `rg` checks over `apps/frontend/src` for forbidden direct calls or leaked infrastructure references.

At minimum, check for:

- `localhost`
- `127.0.0.1`
- `:8000`
- `:8001`
- `:8002`
- `:8011`
- `:8012`
- `:8013`
- `:8020`
- `:8021`
- `:8030`
- `:8040`
- `:8050`
- `red-provider-service`
- `green-provider-service`
- `blue-provider-service`
- `davinci.softlab.ntua.gr`
- `amqp://`
- `rabbitmq`
- `/api/red-provider`
- `/api/green-provider`
- `/api/blue-provider`

Use safe `rg -e` arguments if shell quoting is fragile.

AI log requirements:

Create this prompt file:

```
docs/ai/prompts/prompt_15_finalization_project_audit.md
```

Create this run report:

```
docs/ai/codex-runs/015-finalization-project-audit.md
```

Update:

```
docs/ai/prompt-log.md
```

Add a new row/entry for run `015`.

Update `docs/final-verification.md` only if the actual verification results materially change from the existing final verification report.

Allowed repository changes in this prompt:

- `docs/ai/prompts/prompt_15_finalization_project_audit.md`
- `docs/ai/codex-runs/015-finalization-project-audit.md`
- `docs/ai/prompt-log.md`
- `docs/final-verification.md` only if verification results materially change
- rendered PlantUML output files only if the repository already expects/permits rendered diagrams; otherwise do not commit rendered image artifacts and document local rendering success in the run report

Do not modify application source, backend source, frontend source, Docker topology, Nginx routes, service schemas, provider adapters, RabbitMQ contracts, business logic, assignment sources, or lecture/course sources.

Final response format:

Return exactly these sections:

Status

Use one of:

- `PASS`
- `PASS_WITH_WARNINGS`
- `FAIL`

GitHub Project and issue changes

State:

- whether old project closure succeeded;

- whether the new project was created;
- new project title and number if available;
- issues created and their numbers;
- whether issues were added to the new project;
- unavailable GitHub project operations, if any.

Phase A audit result

Answer:

1. Is the implementation complete enough for the original course submission?
2. Does it match `AGENTS.md` and `docs/specs/SPECIFICATIONS.md`?
3. Are all services present and correctly separated by responsibility?
4. Does the frontend call only gateway paths?
5. Are provider services kept internal and not exposed directly to the browser?
6. Is RabbitMQ used only for background/event flows, not direct frontend actions?
7. Is the one-PostgreSQL-container-per-stateful-service rule preserved?
8. Are final docs and final PlantUML source diagrams present and coherent?
9. Are AI logs complete and consistent?
10. Are there any blockers before final submission?
11. Are there any non-blocking warnings that should be documented?
12. Is Prompt 14's `COMPLETED_WITH_WARNINGS` status still acceptable?

Commands run

List each command and pass/fail/skipped/unavailable status.

Files changed

List exact changed files.

Warnings

List only real, evidence-grounded warnings.

Blockers

Say `No blockers` only if the evidence supports it.

Next phase

State that Phase B should be the next project item:

```
Phase B - Configure real environment variables and verify live functionality
```

Do not begin Phase B in this run.

1. Can JWT secret be something like this?
1XC4Q1QTcJ8UvyK5EmdLALV4dikA2DKBynPz60BM7v5
M6CrtTMvcWcUxK6JC

2. Έβαλα password

Για το 3 και μετά , θέλει να κάνω configure oAuth screen consent. Πες μου τι να βάλω:

1. Does this screenshot help?
2. I have connected via my vpn in this page, where the swagger of apis is. Access it and tell me if we have something useful there (https://davinci.softlab.ntua.gr/saas26/swagger#/provider-docs/list_provider_specs_docs_get). It is only accessible via my vpn.

Επικολλημένο κείμενο....
Έγγραφο

Κατέβασα αυτό το αρχείο από εκεί. Είναι χρήσιμο (pasted json);

Διόρθωσα τον κωδικό και έκανα αυτό. Λειτούργησε;

Τα έβαλα. Τι να ρυθμίσω ως api key τώρα και πού;

Τα έβαλα. Πώς το τρέχω τώρα;

Το έτρεξα για όλους.

Σκέφτηκε για 16s ›

Προτεινόμενη προσέγγιση

Θα ακολουθούσα αυτήν:

```
Lightweight scheduler-service as central control-plane
+
service-owned internal job endpoints
+
optional service-owned workers only where needed later
```

Δηλαδή, για το Phase C, **όχι pure per-service loops** και **όχι full business orchestrator**.

Η πρώτη υλοποίηση να είναι:

```
scheduler-service
  -> calls internal job endpoints on responsible services
  -> records job schedules, runs, status, errors
  -> exposes operator-visible scheduler status
```

Τα services παραμένουν ιδιοκτήτες της δουλειάς:

```
catalog-service performs catalog import
reservation-service publishes reservation outbox
analytics-service drains events and refreshes aggregates
billing-service drains billing events and creates charges
provider services perform provider-local sync if needed
```

Γιατί αυτή είναι η καλύτερη προσέγγιση εδώ

1. Είναι πιο καθαρό για demo/εξέταση

Ο καθηγητής θα δει ένα σαφές concept:

```
There is a scheduler/control-plane for background jobs.
```

Αυτό εξηγείται εύκολα. Έχεις ένα σημείο για:

- job definitions,
- run history,
- failures,
- retries,

- operator visibility.

Με pure per-service loops, η λογική είναι πιο “κρυμμένη” σε πολλά containers.

---

2. Δεν γίνεται god orchestrator

Ο scheduler **δεν** θα κάνει business logic.

Δεν θα:

- κάνει provider normalization,
- γράφει catalog rows,
- δημιουργεί charges,
- υπολογίζει analytics,
- γράφει σε βάσεις άλλων services,
- αποφασίζει business rules.

Θα κάνει μόνο:

```
trigger job
record status
retry if configured
show run history
```

Άρα δεν καταστρέφει microservice ownership.

---

3. Είναι πιο ασφαλές implementation-wise

Το repo ήδη έχει service endpoints και Docker Compose. Το πιο απλό incremental βήμα είναι να προσθέσουμε:

```
scheduler-service
scheduler-postgres
internal job endpoints
operator scheduler view/API
```

χωρίς να ξανασχεδιάσουμε όλο το event system.

Συγκεκριμένη αρχιτεκτονική

Νέα service/container layer

Προσθέτουμε:

```
scheduler-service
scheduler-postgres
```

Το `scheduler-service` έχει δική του βάση για:

```
scheduled_jobs
job_runs
job_attempts / errors
```

Δεν χρησιμοποιεί τις βάσεις άλλων services.

Internal job endpoints

Κάθε υπεύθυνο service αποκτά internal endpoint, π.χ.:

```
POST /internal/jobs/run
```

ή πιο συγκεκριμένα:

```
POST /internal/jobs/sync
POST /internal/jobs/import
POST /internal/jobs/drain-events
POST /internal/jobs/refresh-aggregates
POST /internal/jobs/publish-outbox
```

Αυτά δεν πρέπει να εκτίθενται στο browser. Είναι Docker-network internal ή protected operator/internal routes.

Ποια jobs να υλοποιηθούν πρώτα

1. Provider sync / catalog import

Καλύτερη ροή:

```
scheduler-service
  -> catalog-service internal job: import all providers
      -> catalog-service calls red/green/blue provider services
          -> provider services call live provider APIs
      -> catalog-service updates aggregated catalog
```

Αυτό είναι καλύτερο από το να καλεί ο scheduler απευθείας red/green/blue APIs.

Ο scheduler απλώς λέει:

```
Run catalog import now.
```

Το catalog-service ξέρει πώς γίνεται.

Suggested interval:

```
every 5 minutes
```

Env:

```env
SCHEDULER_CATALOG_IMPORT_INTERVAL_SECONDS=300
```

---

2. Reservation outbox publish

Ροή:

```
scheduler-service
  -> reservation-service internal job: publish pending outbox events
      -> reservation-service publishes pending events to RabbitMQ
```

Suggested interval:

```
every 30 seconds
```

Env:

```env
SCHEDULER_RESERVATION_OUTBOX_INTERVAL_SECONDS=30
```

---

3. Analytics event drain + aggregate refresh

Ροή:

```
scheduler-service
  -> analytics-service internal job: drain usage events
  -> analytics-service internal job: refresh aggregates
```

Intervals:

```
drain events: every 30 seconds
refresh aggregates: every 5 minutes
```

Env:

```env
SCHEDULER_ANALYTICS_DRAIN_INTERVAL_SECONDS=30
SCHEDULER_ANALYTICS_REFRESH_INTERVAL_SECONDS=300
```

---

4. Billing event drain

Ροή:

```
scheduler-service
  -> billing-service internal job: drain billing events
      -> billing-service creates charges/invoice lines as appropriate
```

Interval:

```
every 30 seconds
```

Env:

```env
SCHEDULER_BILLING_DRAIN_INTERVAL_SECONDS=30
```

Τι να μη βάλουμε τώρα

Δεν θα έβαζα ακόμα:

```
red-provider-worker
green-provider-worker
blue-provider-worker
```

εκτός αν το repo ήδη έχει ξεκάθαρη provider-local sync function που αξίζει να τρέχει ξεχωριστά.

Για πρώτη έκδοση, το καλύτερο είναι:

```
catalog-service owns provider import orchestration
scheduler-service only triggers catalog import
```

Αν αργότερα δούμε ότι χρειάζεται provider-local scheduled snapshots, το προσθέτουμε.

HTTP trigger ή RabbitMQ command;

Υπάρχουν δύο επιλογές.

Επιλογή 1 - scheduler calls internal HTTP endpoints

```
scheduler-service -> POST http://catalog-service:8000/internal/jobs/import
```

Πλεονεκτήματα

- απλούστερο,
- deterministic,
- εύκολο debug,
- ταιριάζει με Docker Compose,
- λιγότερος κώδικας,
- πιο κατάλληλο για το project τώρα.

Μειονεκτήματα

- πιο άμεσο coupling,
- αν το service είναι κάτω, ο scheduler πρέπει να γράψει failed run και να retry later.

Επιλογή 2 - scheduler publishes RabbitMQ job commands

```
scheduler-service -> RabbitMQ -> catalog-worker
```

Πλεονεκτήματα

- πιο event-driven,
- καλύτερη αποσύζευξη,
- πιο production-like.

Μειονεκτήματα

- περισσότερα contracts,
- command queues,
- consumers,
- idempotency,
- περισσότερη πολυπλοκότητα.

Recommendation

Για τώρα:

```
Use internal HTTP job endpoints.
```

Το RabbitMQ να μείνει για domain/events propagation, όχι για scheduler commands στην πρώτη φάση.

Αυτό είναι πιο καθαρό, μικρότερο και λιγότερο risky.

Πώς θα φαίνεται συνολικά

```
Operator UI
   |
   v
Nginx gateway
   |
   v
scheduler-service
   |
   +--> catalog-service /internal/jobs/import
   |
   +--> reservation-service /internal/jobs/publish-outbox
   |
   +--> analytics-service /internal/jobs/drain-events
   |
   +--> analytics-service /internal/jobs/refresh-aggregates
   |
   +--> billing-service /internal/jobs/drain-events
```

Και παράλληλα:

```
reservation-service -> RabbitMQ -> analytics-service / billing-service
```

Άρα κρατάμε:

- synchronous user actions,
- async event propagation,
- scheduled background triggering,
- service-owned execution.

Τεχνικές αποφάσεις

Scheduler implementation

Θα χρησιμοποιούσα απλό Python FastAPI service με:

```
FastAPI app
background scheduler loop
scheduler-postgres
SQLAlchemy/SQLModel
httpx for internal calls
```

Δεν είναι ανάγκη να βάλουμε Celery/Airflow/Temporal.

Για scheduling engine:

Προτίμηση

```
APScheduler
```

Γιατί δίνει καθαρό interval scheduling, job IDs, misfire handling και είναι γνωστό Python tool.

Αν θέλουμε ultra-minimal:

```
asyncio loop
```

Αλλά για πιο “μηχανική” λύση, APScheduler είναι καλύτερο.

Minimum data model

`scheduled_jobs`

```
id
name
target_service
target_url
method
interval_seconds
enabled
timeout_seconds
max_retries
created_at
updated_at
```

`job_runs`

```
id
job_id
status
started_at
finished_at
duration_ms
http_status
error_message
attempt_count
```

Status values:

```
PENDING
RUNNING
SUCCESS
FAILED
SKIPPED
```

Operator controls

Στο operator dashboard αξίζει να εμφανιστούν:

```
Scheduled Jobs
- name
- enabled/disabled
- interval
- last run status
- last run time
- next run time
- manual run button
```

Για το Phase C implementation, μπορεί να είναι minimal.

Τελική επιλογή

Η προσέγγιση που προτείνω είναι:

```
Central scheduler-service with scheduler-postgres,
using internal HTTP job triggers,
while each target service owns and executes its own job logic.
```

Όχι:

```
central orchestrator with business logic
```

Όχι:

```
independent loops hidden inside every service only
```

Πώς να το διατυπώσουμε για Codex

Το επόμενο prompt δεν πρέπει να ζητήσει implementation αμέσως. Πρώτα design-only:

```
Design Phase C scheduler/control-plane implementation only.

We choose a hybrid architecture:
- central scheduler-service as control-plane;
- scheduler-postgres for job definitions and run history;
- internal HTTP job endpoints on responsible services;
- service-owned execution and data ownership;
- no business logic inside scheduler-service;
- no RabbitMQ for live user actions;
- no central database access to other service DBs.

Do not implement yet.

Produce:
1. proposed service/container changes;
2. exact jobs and intervals;
3. internal endpoints to add per service;
4. scheduler data model;
5. operator UI/API impact;
6. risks and mitigations;
7. affected files;
8. validation plan.

Update only AI logs:
- docs/ai/prompts/prompt_17_scheduler_design.md
- docs/ai/codex-runs/017-scheduler-design.md
- docs/ai/prompt-log.md

Do not modify application source code.
```

Μετά, αφού δούμε το design output, δίνουμε implementation prompt.

Έδωσα αυτό το prompt και αυτή ήταν η έξοδος.
Δες το από το repo με το connector και πες μου πώς συνεχίζουμε.

Done. I created the Phase C scheduler/control-plane design only, with no application source changes.

Changed files:
- `docs/ai/prompts/prompt_17_scheduler_design.md`
- `docs/ai/codex-runs/017-scheduler-design.md`

Εμφάνιση περισσότερων

Έγινε λήψη της απάντησης ης εφαρμογής

Σκέφτηκε για 36s

Έλεγχος του design

Το design που παρήγαγε ο Codex είναι **σωστό και εφαρμόσιμο**. Κρατάει τη hybrid προσέγγιση που συμφωνήσαμε:

```
central scheduler-service as control-plane
+
service-owned business execution
```

Το αρχείο δηλώνει ρητά ότι το `scheduler-service` δεν είναι business orchestrator, δεν διαβάζει/γράφει βάσεις άλλων services, και εκτελεί jobs καλώντας Docker-internal HTTP endpoints των υπεύθυνων services.

017-scheduler-design

Τι επιβεβαιώνεται από το design

Θέμα	Απόφαση	Αξιολόγηση
Νέα service/container	`scheduler-service` + `scheduler-postgres`	Σωστό, αλλά είναι SPEC amendment γιατί προστίθεται νέο service. 017-scheduler-design
Ρόλος scheduler	Control-plane μόνο, όχι business orchestrator	Σωστό. 017-scheduler-design
Εκτέλεση jobs	Internal HTTP endpoints στα responsible services	Σωστό και απλό για Docker Compose. 017-scheduler-design
Ασφάλεια internal endpoints	`X-Scheduler-Token`, `X-Job-Run-Id`, redacted summaries	Σωστό. 017-scheduler-design

Θέμα	Απόφαση	Αξιολόγηση
Jobs	Catalog import, analytics drain/refresh, billing drain/outbox, reservation outbox, invoice preparation	Καλό αρχικό set. 017-scheduler-design
Operator UI/API	`/api/scheduler/jobs`, run history, pause/resume, health	Χρήσιμο για τελική επίδειξη. 017-scheduler-design
Risks	Duplicate execution, provider rate pressure, secret leakage, data ownership drift	Καλύπτονται σωστά. 017-scheduler-design
Acceptance criteria	No cross-service DB writes, no RabbitMQ for live actions, no secret logging	Σωστά. 017-scheduler-design

Η μόνη σημαντική απόφαση πριν implementation

Πρέπει να αποδεχτούμε επίσημα ότι το canonical service set επεκτείνεται με:

```
scheduler-service
scheduler-postgres
```

Το ίδιο το design το αναφέρει ως pre-implementation confirmation: να επιβεβαιωθεί ότι αυτά γίνονται αποδεκτά ως SPEC amendment.

017-scheduler-design

Εγώ θα το αποδεχόμουν. Είναι τεχνικά σωστό και πλέον justified από το Phase C hardening scope.

Πώς συνεχίζουμε

Δεν θα έδινα ένα τεράστιο implementation prompt. Θα το σπάσουμε σε μικρά prompts.

Βήμα 0 - Checkpoint

Πριν από νέο Codex prompt:

1. Κάνε ένα commit ή τουλάχιστον βεβαιώσου ότι το working tree είναι καθαρό/κατανοητό.
2. Μην ανακατέψεις Phase A/B/C changes χωρίς checkpoint.

Προτεινόμενο commit message:

```
feat(scheduler): add Phase C scheduler design documentation
```

ή αν κάνεις commit όλα τα Phase A/B/C docs/fixes μαζί:

```
chore(project): record finalization audit and live provider verification
```

Βήμα 1 - SPEC amendment μόνο

Πρώτα να ενημερώσει τα docs/specs ώστε να μη φαίνεται ότι το `scheduler-service` παραβιάζει το canonical service set.

Prompt για Codex

```
Continue Phase C only.

Task: record the accepted scheduler/control-plane SPEC amendment.

Context:
- Phase B passed with warnings and issue #16 is closed.
- Phase C scheduler design was completed in `docs/ai/codex-runs/017-scheduler-desig
- We accept the hybrid approach:
  - `scheduler-service` as central control-plane;
  - `scheduler-postgres` as scheduler-owned database;
  - service-owned execution through internal HTTP job endpoints;
  - no business logic inside scheduler-service;
  - no cross-service DB access;
  - no RabbitMQ for live user actions.

Do not implement source code yet.

Update only documentation and AI logs:
- `docs/specs/SPECIFICATIONS.md`
- `docs/architecture-summary.md`
- `docs/ai/prompts/prompt_18_scheduler_spec_amendment.md`
- `docs/ai/codex-runs/018-scheduler-spec-amendment.md`
- `docs/ai/prompt-log.md`

If needed, update `README.md` only to mention the accepted scheduler-service additi

Do not modify application code.
Do not modify Docker Compose yet.
Do not modify frontend code.
Do not start scheduler implementation.
Do not touch Phase D.

Also update Phase C issue/project status if available:
- keep Phase C issue open/In Progress;
```

```
- do not close it yet.

Return:
## Status
## Spec amendment summary
## Files changed
## Warnings
## Blockers
## Next step
```

Βήμα 2 - Scheduler service scaffold

Μετά το spec amendment, δίνουμε μικρό implementation prompt μόνο για το skeleton.

Να δημιουργήσει:

```
services/scheduler-service/
scheduler-postgres
docker-compose entries
gateway route /api/scheduler/*
health endpoint
operator JWT protection skeleton
scheduler DB connection
basic tables/models
tests for health/config/security
```

Όχι ακόμα πραγματικά jobs.

Βήμα 3 - Internal job endpoints

Μετά υλοποιούμε ανά service:

1. `reservation-service`: outbox publish job.
2. `analytics-service`: consume events + refresh aggregates.
3. `billing-service`: drain events + publish outbox.
4. `catalog-service`: import all/providers.

Να μη γίνουν όλα με ένα prompt αν δεν χρειάζεται.

Βήμα 4 - Scheduler executes jobs

Αφού υπάρχουν endpoints, τότε βάζουμε:

- due-job loop,

- manual run,
- run history,
- retries,
- lock handling,
- safe response summaries.

Βήμα 5 - Operator UI

Τελευταίο:

- scheduler status cards,
- job list,
- run history,
- manual run buttons,
- pause/resume.

Προτεινόμενη σειρά Phase C

Step	Prompt	Περιεχόμενο
18	SPEC amendment	Αναγνώριση `scheduler-service` ως accepted architecture extension
19	Scheduler scaffold	Service, DB, Docker, gateway, health, models
20	Internal job endpoints	Service-owned endpoints στα catalog/analytics/billing/reservation
21	Scheduler execution	Job loop, manual runs, retries, run history
22	Operator UI	Scheduler dashboard/controls
23	Verification	Full Phase C validation και close issue

Αυτή είναι η πιο ασφαλής πορεία.

Δώσε μου ένα prompt για όλα μαζί, για όλη τη φάση c.

Σκέφτηκε για 13s ›

Ναι. Παρακάτω είναι **ένα ενιαίο prompt για όλη τη Phase C scheduler/control-plane υλοποίηση**, με spec/docs/code/tests/UI/verification μαζί.

```
You are working in:

ntua-el21026/NTUA-saasPlug

Task type:

PHASE C - SCHEDULER / CONTROL-PLANE IMPLEMENTATION

Goal:

Implement the accepted hybrid scheduler/control-plane architecture for saasPlug.

This is a full Phase C implementation prompt. Complete the scheduler/control-plane

Do not start Phase D root-level broad test-suite work.
Do not implement deployment or Vercel.
Do not commit `.env`.
Do not print secrets.
Do not expose provider API keys, JWT secrets, Google secrets, RabbitMQ passwords, c
All repository content must remain in English.

Canonical context:

Read first:

- AGENTS.md
- docs/specs/SPECIFICATIONS.md
- docs/architecture-summary.md
- docs/final-verification.md
- docs/submission-checklist.md
- docs/ai/prompt-log.md
- docs/ai/codex-runs/016-live-env-verification.md
- docs/ai/codex-runs/017-scheduler-design.md
- docker-compose.yml
- deploy/nginx/default.conf
- .env.example

Important previous state:

- Phase A completed.
- Phase B passed with warnings.
- Live provider credentials and Bearer auth now work.
- Provider API live read-only checks passed:
  - redPlug list/detail: 200
  - greenPlug list/detail: 200
  - bluePlug list/status: 200
- Phase B issue is closed/done.
- Phase C scheduler design exists in `docs/ai/codex-runs/017-scheduler-design.md`.

Accepted Phase C architecture:
```

```
Implement the hybrid design:

- Add `scheduler-service` as a lightweight central control-plane.
- Add `scheduler-postgres` as scheduler-owned database.
- Scheduler stores only scheduler metadata:
  - job definitions;
  - run history;
  - run status;
  - safe summaries;
  - locks/metadata.
- Scheduler does not contain business logic.
- Scheduler does not read or write any other service database.
- Actual job execution remains service-owned.
- Scheduler triggers responsible services through Docker-internal HTTP endpoints.
- Internal job endpoints require scheduler authentication.
- Live user actions remain synchronous HTTP and must not move to RabbitMQ.
- RabbitMQ remains for background/event propagation only.
- No service may write to another service database.

Add SPEC amendment:

Update canonical documentation to include:

- `scheduler-service`;
- `scheduler-postgres`;
- scheduler/control-plane role;
- internal job endpoints;
- operator scheduler controls;
- no business orchestration and no cross-service DB access.

Minimum files to update for docs/specs:

- docs/specs/SPECIFICATIONS.md
- docs/architecture-summary.md
- README.md if needed
- docs/final-verification.md
- docs/submission-checklist.md

AI log requirements:

Create:

- docs/ai/prompts/prompt_18_scheduler_control_plane_implementation.md
- docs/ai/codex-runs/018-scheduler-control-plane-implementation.md

Update:

- docs/ai/prompt-log.md

If the GitHub Phase C issue/project item is available:

- move Phase C to In Progress at the start;
```

- close/move to Done only if the final Phase C validation passes or passes with acc
- do not touch Phase D except to mention it as next.

Implementation requirements:

1. Add scheduler service

Create:

- services/scheduler-service/

Use the project’s existing FastAPI/Python conventions.

The service must include at minimum:

- FastAPI app;
- `/health`;
- operator-protected scheduler APIs;
- scheduler DB configuration;
- SQLAlchemy/SQLModel models or repository layer consistent with existing backend s
- scheduler loop or background runner;
- HTTP client for internal service calls;
- safe redaction utilities;
- tests.

Suggested scheduler tables:

- `scheduled_jobs`
- `job_runs`
- optional `scheduler_events` if practical

Minimum `scheduled_jobs` fields:

- id
- job_key
- display_name
- description
- target_service
- target_method
- target_url
- schedule_type: interval / cron / manual
- interval_seconds
- cron_expression
- timezone
- enabled
- request_body
- timeout_seconds
- max_retries
- retry_backoff_seconds
- jitter_seconds
- max_concurrency
- next_run_at

- last_run_id
- last_status
- created_at
- updated_at

Minimum `job_runs` fields:

- id
- job_id
- job_key
- trigger_type: scheduled / manual / retry
- triggered_by_user_id nullable
- triggered_by_email nullable
- status: queued / running / succeeded / failed / timed_out / skipped / cancelled
- scheduled_for
- started_at
- finished_at
- duration_ms
- attempt
- correlation_id
- scheduler_instance_id
- target_http_status
- response_summary
- error_type
- error_message
- lock_expires_at
- created_at

2. Add scheduler environment variables

Add placeholders/defaults to `.env.example` and `docker-compose.yml`.

Required variables:

- SCHEDULER_DATABASE_URL
- SCHEDULER_TICK_SECONDS
- SCHEDULER_MAX_CONCURRENT_JOBS
- SCHEDULER_INSTANCE_ID
- SCHEDULER_INTERNAL_TOKEN
- SCHEDULER_HTTP_TIMEOUT_SECONDS
- CATALOG_SERVICE_URL
- ANALYTICS_SERVICE_URL
- BILLING_SERVICE_URL
- RESERVATION_SERVICE_URL

Use safe placeholders only. Do not add real secrets.

If `.env` does not contain `SCHEDULER_INTERNAL_TOKEN`, do not modify `.env`. Use C

3. Add Docker Compose integration

Add:

- `scheduler-service`
- `scheduler-postgres`
- scheduler volume
- healthcheck
- dependency wiring

Preserve:

- one PostgreSQL container per stateful service;
- existing service containers;
- existing RabbitMQ usage;
- existing gateway behavior.

4. Add Nginx gateway route

Expose only operator scheduler APIs through gateway:

- `/api/scheduler/*`

Do not expose service-owned `/internal/jobs/*` routes through Nginx.

5. Add scheduler APIs

Expose operator-only APIs:

- `GET /api/scheduler/jobs`
- `GET /api/scheduler/jobs/{job_key}`
- `PATCH /api/scheduler/jobs/{job_key}`
- `POST /api/scheduler/jobs/{job_key}/run`
- `POST /api/scheduler/jobs/{job_key}/pause`
- `POST /api/scheduler/jobs/{job_key}/resume`
- `GET /api/scheduler/runs`
- `GET /api/scheduler/runs/{run_id}`
- `GET /api/scheduler/health`

Access rules:

- OPERATOR only.
- EV_USER and PROVIDER_ADMIN must be denied.
- Validate the same JWT style used by other services.
- Do not log tokens.

6. Seed initial scheduler jobs

Seed deterministic default job definitions:

- `catalog.import.all`
  - target: catalog-service
  - endpoint: `POST /internal/jobs/catalog/imports/all`
  - every 15 minutes
  - timeout 180s

  - retries 1

- `analytics.consume-events`
  - target: analytics-service
  - endpoint: `POST /internal/jobs/events/consume`
  - every 60 seconds
  - timeout 30s
  - retries 2

- `analytics.refresh-aggregates`
  - target: analytics-service
  - endpoint: `POST /internal/jobs/aggregates/refresh`
  - every 5 minutes
  - timeout 90s
  - retries 1

- `billing.drain-events`
  - target: billing-service
  - endpoint: `POST /internal/jobs/events/drain`
  - every 60 seconds
  - timeout 30s
  - retries 2

- `billing.publish-outbox`
  - target: billing-service
  - endpoint: `POST /internal/jobs/outbox/publish-pending`
  - every 60 seconds
  - timeout 30s
  - retries 2

- `reservation.publish-outbox`
  - target: reservation-service
  - endpoint: `POST /internal/jobs/outbox/publish-pending`
  - every 60 seconds
  - timeout 30s
  - retries 2

- `billing.prepare-invoices`
  - target: billing-service
  - endpoint: `POST /internal/jobs/invoices/prepare`
  - daily at 02:30 Europe/Athens
  - timeout 180s
  - retries 1
  - must prepare candidates/summaries only; operator still issues invoices explicit

Also seed manual-only jobs:

- `catalog.import.red`
- `catalog.import.green`
- `catalog.import.blue`
- `analytics.refresh-aggregates.manual`
- `billing.drain-events.manual`

7. Add internal service-owned job endpoints

Add internal endpoints to responsible services.

All internal endpoints must:

- require `X-Scheduler-Token`;
- require `X-Job-Run-Id`;
- accept optional `X-Correlation-Id`;
- return compact safe summaries only;
- not return raw provider response bodies;
- not log secrets;
- be idempotent for repeated `X-Job-Run-Id` where practical;
- remain unavailable directly from frontend.

Required endpoints:

catalog-service:

- `POST /internal/jobs/catalog/imports/all`
- `POST /internal/jobs/catalog/imports/providers/{provider_code}`
- `GET /internal/jobs/catalog/freshness`

analytics-service:

- `POST /internal/jobs/events/consume`
- `POST /internal/jobs/aggregates/refresh`
- `GET /internal/jobs/freshness`

billing-service:

- `POST /internal/jobs/events/drain`
- `POST /internal/jobs/outbox/publish-pending`
- `POST /internal/jobs/invoices/prepare`
- `GET /internal/jobs/freshness`

reservation-service:

- `POST /internal/jobs/outbox/publish-pending`
- `GET /internal/jobs/freshness`

Do not add provider-service scheduled endpoints unless required by existing code. F

8. Scheduler execution behavior

Implement:

- due-job selection;
- next-run calculation;
- interval jobs;
- cron/daily job support if practical;

- manual run creation;
- pause/resume;
- retry with bounded backoff;
- per-job timeout;
- `max_concurrency=1` per job key;
- safe run summaries;
- lock expiry / crashed-run recovery if practical;
- no duplicate overlapping runs.

If cron support would be too large, implement interval/manual jobs first and repres

9. Frontend operator UI

Update the existing operator UI.

Add or extend pages/components under the operator area:

- scheduler health card;
- scheduled jobs list;
- job status labels;
- last run / next run;
- manual run button;
- pause/resume buttons with confirmation;
- run history table;
- safe run detail view or expandable row.

Recommended placement:

- extend `/operator/system` with scheduler health and recent failed jobs;
- extend `/operator/sync` with manual job controls and run history.

Frontend rules:

- browser calls only gateway `/api/...` routes;
- no direct internal service URLs;
- no RabbitMQ/browser access;
- no provider API calls from browser.

10. Tests

Add/update focused tests.

Required:

scheduler-service tests:

- health;
- operator authorization;
- EV/provider denial;
- job seed existence;
- interval next-run calculation;
- due-job selection;

```
- manual run creation;
- pause/resume;
- response redaction;
- failed internal call records failed run;
- successful internal call records succeeded run.

Service internal endpoint tests:

- `X-Scheduler-Token` required;
- `X-Job-Run-Id` required;
- safe summary response;
- no secret/raw provider body returned;
- existing service behavior remains passing.

Frontend tests:

- scheduler operator routes/components render;
- EV/provider users cannot access scheduler UI;
- operator can view scheduler cards/list;
- API calls are gateway-relative.

Existing tests must continue to pass.

11. Validation commands

Run as many as available, and report unavailable tools clearly.

Required:

```bash
git status --short
git diff --check
docker compose config --quiet
```
```

Frontend:

Bash

```bash
cd apps/frontend
npm run test
npm run build
cd ../..
```

Backend/service tests:

- run scheduler-service tests;
- run touched service tests;
- run existing provider/auth/catalog/reservation/analytics/billing tests as practical.

Shared events:

Bash

```
PYTHONPATH=packages/events python -m pytest packages/events/tests -q
```

If the known Python/pytest capture issue occurs, retry:

Bash

```
PYTHONPATH=packages/events python -m pytest packages/events/tests -q -s
```

Docker/runtime:

Bash

```
docker compose up -d --build
docker compose ps
```

Gateway health:

Bash

```
curl -fsS http://localhost:8080/health
curl -fsS http://localhost:8080/api/auth/health
curl -fsS http://localhost:8080/api/providers/health
curl -fsS http://localhost:8080/api/catalog/health
curl -fsS http://localhost:8080/api/status/health
curl -fsS http://localhost:8080/api/reservations/health
curl -fsS http://localhost:8080/api/analytics/health
curl -fsS http://localhost:8080/api/billing/health
curl -fsS http://localhost:8080/api/scheduler/health
```

Authenticated smoke:

- EV login works;
- provider-admin login works;
- operator login works;
- operator can call scheduler API;
- EV and provider-admin cannot call scheduler API.

Scheduler smoke:

- manually trigger `catalog.import.all`;
- manually trigger `analytics.consume-events`;
- manually trigger `analytics.refresh-aggregates`;
- manually trigger `billing.drain-events`;
- manually trigger `reservation.publish-outbox`;
- verify run records and safe summaries;

- verify no raw secrets are stored or printed.

Automatic schedule smoke:

- if feasible, temporarily use short intervals through test/local config only;
- verify at least one scheduled job is picked up automatically;
- restore defaults.

Cleanup:

```bash
docker compose down -v
docker compose ps -a
```

Static checks:

- scan that frontend uses only `/api/...`;
- scan that `/internal/jobs/*` is not exposed through Nginx;
- scan that `.env` is not tracked;
- scan for obvious committed secrets;
- verify `openapi.json` remains untracked unless already tracked intentionally.

12. Documentation updates

Update:

- README.md
- docs/specs/SPECIFICATIONS.md
- docs/architecture-summary.md
- docs/final-verification.md
- docs/submission-checklist.md
- docs/diagrams/final/* if diagrams materially change
- docs/diagrams/final/README.md if needed

Diagrams:

If architecture/deployment/component/sequence diagrams need scheduler updates, update PlantUML source and validate:

```bash
plantuml -checkonly docs/diagrams/final/*.puml
```

If `-checkonly` is unsupported, render as validation and report the limitation.

13. GitHub issue/project handling

Find the Phase C scheduler issue by title/project context. Do not rely blindly on issue number.

At the start:

- move Phase C scheduler issue to In Progress if possible.

At the end:

- close/move to Done only if Phase C validation passes or passes with accepted warnings.

Do not close or modify Phase D issue.

14. Allowed changes

Allowed:

- scheduler-service source/tests;
- touched service internal job endpoints/tests;
- frontend operator scheduler UI/tests;
- Docker Compose;
- Nginx gateway;
- `.env.example`;
- documentation/specs/diagrams;
- AI prompt/run logs.

Not allowed:

- `.env`;
- deployment/Vercel;
- broad root-level Phase D tests;
- unrelated refactors;
- moving live user actions to RabbitMQ;
- cross-service DB writes;
- provider API secrets in docs/logs;
- Greek repository content.

Final response format:

Return exactly:

Status

PASS / PASS_WITH_WARNINGS / FAIL

Architecture implemented

Briefly describe scheduler-service, scheduler-postgres, internal job endpoints, and operator UI.

Jobs implemented

Table of job keys, schedule/manual, target service, endpoint.

Validation

Commands run and pass/fail/skipped/unavailable status.

Files changed

Exact file list grouped by area.

Warnings

Only real warnings.

Blockers

No blockers, or precise blocker list.

GitHub issue/project status

What happened to the Phase C issue/project item.

Next phase

State whether Phase D should start next and what remains.

```
Σημαντικό: επειδή αυτό είναι μεγάλο scope, το `PASS_WITH_WARNINGS` θα είναι αποδεκ
```

This is the output of Codex.
Inspect it (via the live connector) and tell me if we are okay for phase D.

Status
PASS_WITH_WARNINGS

Architecture implemented
Implemented Phase C only: `scheduler-service` control plane, `scheduler-postgres`, scheduler-owned metadata

Εμφάνιση περισσότερων

Έγινε λήψη της απάντησης ης εφαρμογής

Έγινε λήψη της απάντησης ης εφαρμογής

Σκέφτηκε για 1m 37s

Συμπέρασμα

Η Phase C είναι **ουσιαστικά επιτυχημένη**, αλλά εγώ **δεν θα πήγαινα ακόμα Phase D** πριν κάνουμε ένα μικρό Phase C hardening fix.

Ο λόγος: ο scheduler/control-plane υλοποιήθηκε σωστά, αλλά από το `nginx/default.conf` φαίνεται ότι τα broad gateway routes πιθανώς εκθέτουν τα `/internal/jobs/*` endpoints μέσω gateway. Τα endpoints έχουν `X-Scheduler-Token`, άρα δεν είναι ανεξέλεγκτα, αλλά το design ζητούσε να **μην είναι καν διαθέσιμα από το gateway/browser path**.

Άρα: **Phase C PASS με ένα μικρό hardening TODO πριν Phase D.**

Τι επιβεβαίωσα από το repo

Περιοχή	Κατάσταση	Evidence
Scheduler service	OK	Το run report λέει ότι προστέθηκε `services/scheduler-service` με FastAPI, persistence, seeded jobs, loop, internal HTTP client, operator APIs και tests. 018-scheduler-control-plane-imp…
Scheduler DB	OK	`docker-compose.yml` έχει `scheduler-postgres` με δικό του volume και DB `scheduler`. docker-compose
Compose wiring	OK	Το gateway εξαρτάται από `scheduler-service`, και το scheduler έχει env vars, service URLs και dependencies προς target services. docker-compose docker-compose
Gateway route	OK για scheduler API	Υπάρχει `/api/scheduler/` route προς `scheduler-service`. default default
Operator protection	OK	Τα scheduler APIs καλούν `ensure_operator(principal)` για jobs/runs/manual run/pause/resume. routes routes
Job definitions	OK	Περιλαμβάνονται catalog import, analytics consume/refresh, billing drain/outbox/invoices, reservation outbox και manual jobs. jobs
Internal endpoint auth	OK εντός service	Τα catalog internal endpoints απαιτούν `X-Scheduler-Token` και `X-Job-Run-Id`. routes Τα analytics internal endpoints κάνουν το ίδιο. routes

Περιοχή	Κατάσταση	Evidence
No cross-service DB	OK by report	Το run report δηλώνει conformance: no cross-service DB access, no scheduler-owned business logic, no live user actions moved to RabbitMQ. 018-scheduler-control-plane-imp…
Validation	Strong	Πέρασαν scheduler/service/frontend/shared events/PlantUML/Docker/gateway/auth/manual-run checks. 018-scheduler-control-plane-imp…
GitHub issue	OK	Phase C issue #18 είναι closed/completed. Phase C - Resolve final warning… Phase D issue #17 μένει open. Phase D - Expand application te…

Το σημείο που θέλει διόρθωση πριν Phase D

Το Nginx έχει broad routes:

Nginx

```
location /api/catalog/ {
    proxy_pass http://catalog_service_upstream/;
}

location /api/analytics/ {
    proxy_pass http://analytics_service_upstream/;
}

location /api/billing/ {
    proxy_pass http://billing_service_upstream/;
}

location /api/reservations/ {
    proxy_pass http://reservation_service_upstream/;
}
```

Αυτό σημαίνει ότι, χωρίς explicit deny/block, ένα URL τύπου:

```
/api/catalog/internal/jobs/catalog/imports/all
```

μπορεί να γίνει proxy προς:

```
catalog-service:/internal/jobs/catalog/imports/all
```

Το `catalog-service` όντως έχει τέτοιο internal endpoint.

routes

Το ίδιο pattern υπάρχει και στο analytics service. routes

Ναι, απαιτείται `X-Scheduler-Token`, αλλά το design/Prompt ζητούσε να μην εκτίθενται τα `/internal/jobs/*` routes από Nginx. Το ίδιο το Phase C report λέει ότι έγινε “Nginx internal job exposure check”, αλλά από το checked `default.conf` δεν βλέπω explicit block.

018-scheduler-control-plane-imp…

Άρα αυτό θέλει μικρό hardening patch.

Επίσης: πρόσθεσε το token στο local `.env`

Το `.env.example` έχει:

```env
SCHEDULER_INTERNAL_TOKEN=change-me-scheduler-token
```

.env

Στο δικό σου `.env` βάλε random value, π.χ. όχι αυτό ακριβώς:

```env
SCHEDULER_INTERNAL_TOKEN=<long-random-local-token>
```

Μην το κάνεις commit.

Αν δεν το βάλεις, το Compose default είναι γνωστό placeholder. Για local demo δεν είναι καταστροφή, αλλά αφού πλέον έχουμε real provider credentials και scheduler endpoints, καλύτερα να το βάλεις σωστά.

Πώς συνεχίζουμε

Βήμα 1 - Μικρό Phase C hardening prompt

Δώσε αυτό στον Codex πριν Phase D:

```
Continue Phase C hardening only.

The Phase C scheduler/control-plane implementation passed, but before Phase D we ne

Issue:
The Nginx gateway has broad routes such as `/api/catalog/`, `/api/analytics/`, `/ap

Tasks:
1. Inspect `deploy/nginx/default.conf` and all service-owned `/internal/jobs/*` rou
2. Add explicit Nginx deny/block rules before the broad service proxy locations so
   - `/api/catalog/internal/`
   - `/api/analytics/internal/`
   - `/api/billing/internal/`
   - `/api/reservations/internal/`
3. Prefer `return 404` or `return 403`; choose one consistently and document it.
4. Do not block `/api/scheduler/*`.
5. Do not change scheduler-service internal Docker calls.
6. Do not modify `.env`.
7. Do not print secrets.
8. Add/update focused tests or smoke checks proving:
   - gateway `/api/catalog/internal/...` is blocked;
   - gateway `/api/analytics/internal/...` is blocked;
   - gateway `/api/billing/internal/...` is blocked;
   - gateway `/api/reservations/internal/...` is blocked;
   - `/api/scheduler/health` still works;
   - scheduler manual run still works through internal Docker service URLs.
9. Update:
   - `docs/ai/codex-runs/018-scheduler-control-plane-implementation.md`
   - `docs/final-verification.md`
   - `docs/submission-checklist.md`
   - `docs/ai/prompt-log.md` only if needed
10. Do not start Phase D.
11. Keep Phase C issue #18 closed unless GitHub process requires a follow-up commer

Return:
## Status
PASS / PASS_WITH_WARNINGS / FAIL

## Hardening change
What changed.

## Validation
Commands and results.

## Files changed
Exact files.

## Warnings
Only real warnings.

## Blockers
```

```
No blockers, or exact blockers.

## Phase D readiness
Whether Phase D can start.
```

Βήμα 2 - Μετά, ναι πάμε Phase D

Αν αυτό περάσει, τότε Phase D είναι το σωστό επόμενο βήμα. Το Phase D issue είναι ήδη ανοιχτό και αφορά root-level/cross-app test coverage.

Phase D - Expand application te…

Τελική απάντηση

Όχι ακόμα Phase D. Είμαστε πολύ κοντά, αλλά πρώτα κάνε το μικρό Nginx hardening για να μπλοκαριστούν gateway paths προς `/internal/jobs/*`.

Μετά από αυτό, ναι: **Phase D είναι το επόμενο σωστό βήμα.**

1. Τι value να βάλω εδώ: SCHEDULER_INTERNAL_TOKEN=<long-random-local-token>

2. Δες το repo μετά την απάντηση του Codex και εάν είμαστε καλά πάμε στη φάση D

Status
PASS

Hardening change

Εμφάνιση περισσότερων

Έγινε λήψη της απάντησης ης εφαρμογής

Σκέφτηκε για 26s

1. Τι value να βάλεις στο `SCHEDULER_INTERNAL_TOKEN`

Βάλε **τυχαίο μεγάλο token**, όχι το default:

```env
SCHEDULER_INTERNAL_TOKEN=change-me-scheduler-token
```

Καλύτερη μορφή: **hex string 64-96 χαρακτήρων**, χωρίς κενά, χωρίς quotes.

Στο WSL τρέξε:

Bash

```
openssl rand -hex 48
```

Θα σου δώσει κάτι σαν:

```
9f0e2c5a7b...<96 hex chars total>
```

Στο `.env` βάλε:

env

```
SCHEDULER_INTERNAL_TOKEN=<το output του openssl rand -hex 48>
```

Μην το βάλεις στο `.env.example`, μην το κάνεις commit, μην το γράψεις σε docs/logs.

---

2. Έλεγχος repo μετά το hardening

Ναι, τώρα είμαστε καλά για Phase D.

Το hardening εφαρμόστηκε σωστά: το Nginx πλέον μπλοκάρει ρητά τα internal job gateway paths με `404` πριν από τα broad proxy routes:

Nginx

```
location ^~ /api/catalog/internal/ {
    return 404;
}

location ^~ /api/reservations/internal/ {
    return 404;
}

location ^~ /api/analytics/internal/ {
    return 404;
}

location ^~ /api/billing/internal/ {
    return 404;
}
```

Ταυτόχρονα, το `/api/scheduler/` παραμένει διαθέσιμο μέσω gateway προς `scheduler-service`.

default

Το updated Phase C run report επίσης καταγράφει ότι πέρασαν:

- gateway block smoke για catalog/analytics/billing/reservations internal routes με `404`,
- `/api/scheduler/health` με `200`,
- scheduler manual run smoke για `billing.prepare-invoices`,
- `nginx -t`,
- Docker cleanup,
- `git diff --check`.

018-scheduler-control-plane-imp…

Η Phase C παραμένει conformant: no cross-service DB access, no scheduler-owned business logic, no live user actions moved to RabbitMQ, service-owned execution, frontend only gateway `/api/...`.

018-scheduler-control-plane-imp…

Verdict

Phase C is complete. We can start Phase D.

Υπάρχουν μόνο προηγούμενες non-blocking limitations: δεν έγινε automatic shortened-interval scheduler-loop smoke και δεν έγινε browser visual QA. Αυτά είναι λογικό να καλυφθούν/ενισχυθούν στη Phase D test expansion.

018-scheduler-control-plane-imp…

Short Phase D prompt

```
Continue with Phase D only.

Goal:
Expand application test coverage for saasPlug without changing architecture or addi

Context:
- Phase A, B, and C are complete.
- Phase C added scheduler-service, scheduler-postgres, internal job endpoints, oper
- Phase D issue #17 is open/Todo.
- Do not touch `.env`.
- Do not print secrets.
- Do not deploy.
- Do not use Vercel.
- Do not change architecture.

Tasks:
1. Inspect current tests across:
```

```
   - apps/frontend/src/test/
   - services/*/tests/
   - packages/events/tests/
2. Create a concise test gap analysis.
3. Add a root-level `tests/` directory only if it is useful for cross-service / Doc
4. Add focused tests for:
   - auth login and role access;
   - provider normalization and Bearer auth behavior;
   - catalog bounds/search/import behavior;
   - status cache and fallback behavior;
   - reservation success/failure/unknown normalization;
   - analytics ingestion/idempotency/aggregate refresh;
   - billing event drain/idempotency/invoice/payment behavior;
   - scheduler operator authorization, manual runs, internal route blocking, and ru
   - gateway smoke paths;
   - frontend role routing and scheduler UI access.
5. Keep service-local tests where they fit best; use root-level tests only for inte
6. Update documentation for how to run tests.
7. Update AI logs:
   - create `docs/ai/prompts/prompt_19_phase_d_test_expansion.md`
   - create `docs/ai/codex-runs/019-phase-d-test-expansion.md`
   - update `docs/ai/prompt-log.md`
8. Update `docs/final-verification.md` and `docs/submission-checklist.md` only if v
9. If all Phase D checks pass or pass with accepted warnings, close Phase D issue #

Validation:
- run frontend tests/build;
- run all touched service tests;
- run scheduler tests;
- run shared event tests;
- run Docker Compose config/startup/health;
- run gateway smoke checks;
- run authenticated EV/provider/operator smoke checks;
- verify internal job gateway routes still return 404;
- run `git diff --check`;
- verify `.env` is not tracked.

Return:
## Status
PASS / PASS_WITH_WARNINGS / FAIL

## Test expansion summary
What tests were added and why.

## Validation
Commands and results.

## Files changed
Exact list.

## Warnings
Only real warnings.
```

```
## Blockers
No blockers, or exact blockers.

## GitHub issue/project status
Whether Phase D issue #17 was closed/moved to Done.

## Final readiness
Whether the repository is ready for final course submission.
```

Αυτή είναι η έξοδος του Codex για το task D.
Δες το live repo με το connector σου και πες μου αν είναι όλα καλά:

Status
PASS

Test expansion summary
Expanded Phase D coverage without architecture or feature changes.

Εμφάνιση περισσότερων

Έγινε λήψη της απάντησης ης εφαρμογής

Σκέφτηκε για 38s

Έλεγχος μετά τη Phase D

Ναι, από το live repo φαίνεται ότι είμαστε σε καλή κατάσταση για τελικό ολικό έλεγχο σε νέα συνομιλία.

Επιβεβαίωσα τα βασικά:

Περιοχή	Κατάσταση	Evidence
Phase D	Ολοκληρωμένη	Το run `019-phase-d-test-expansion.md` καταγράφει ότι η Phase D πέρασε και ότι η συνολική verified suite έχει **184 tests**. 019-phase-d-test-expansion

Περιοχή	Κατάσταση	Evidence
Comprehensive closure	Ολοκληρωμένη	Το run `020-comprehensive-test-expansion-and-project-closure.md` επιβεβαιώνει backend, frontend, shared events, gateway checks, Compose, authenticated smokes, OAuth configured check, scheduler smoke, `.env` untracked, και project closure. 020-comprehensive-test-expansio…
Phase D issue	Closed	Issue #17 είναι closed με `state_reason: completed`. Phase D - Expand application te…
Final verification	Submission-ready	Το `docs/final-verification.md` καταλήγει ότι το repo είναι έτοιμο για τελική υποβολή για το implemented local MVP, με μόνο documented local MVP limitations. final-verification
Remaining warning	Non-blocking	Το μόνο πραγματικό warning είναι transient PostgreSQL connection-refused σε πρώτο full-stack startup, που λύθηκε με targeted restart και όλα τα containers έγιναν healthy. 020-comprehensive-test-expansio…

Άρα: **ναι, μπορούμε να πάμε σε νέα συνομιλία** για καθαρό τελικό review από την αρχή.

Prompt για νέα συνομιλία με ChatGPT

Κάνε copy-paste το παρακάτω ως πρώτο μήνυμα στη νέα συνομιλία. Μετά δώσε connector access στο repo `ntua-el21026/NTUA-saasPlug`.

```
We are continuing the NTUA ECE course project “Technologies of Software Services /

Discussion with me can be in Greek, but all repository files, documentation, code,

I will provide access to the GitHub repository through the available connector. You

Repository target:

ntua-el21026/NTUA-saasPlug

Project name:

saasPlug

Course context:

This is an NTUA ECE semester project for “Technologies of Software Services / Τεχνο
```

The system supports:

- EV users:
  - register/login;
  - search charging points;
  - view map/list/details;
  - check charging-point status;
  - create reservations;
  - view own reservation history;
  - manage profile/session.

- Charging infrastructure providers:
  - redPlug;
  - greenPlug;
  - bluePlug;
  - provider admins view their own points/analytics/invoices;
  - provider admins export usage logs;
  - provider admins pay issued invoices through mock payment flow.

- saasPlug operator:
  - manage providers;
  - view global analytics;
  - manage pricing;
  - run sync/control-plane operations;
  - issue invoices;
  - inspect system status;
  - inspect scheduler/control-plane jobs and run history.

Provider APIs:

The assignment integrates three hypothetical provider APIs:

- redPlug
- greenPlug
- bluePlug

Their official sources are under:

- docs/assignment/providers/yaml/redPlug.yaml
- docs/assignment/providers/yaml/greenPlug.yaml
- docs/assignment/providers/yaml/bluePlug.yaml
- docs/assignment/providers/postman_collections/redPlug.postman_collection.json
- docs/assignment/providers/postman_collections/greenPlug.postman_collection.json
- docs/assignment/providers/postman_collections/bluePlug.postman_collection.json

Important provider-auth context:

During live Phase B verification, the official Postman collections and real provide

Authorization: Bearer <apiKey>

not plain:

```
authorization: <apiKey>

The implementation was updated so provider services send Bearer auth by default thr

PROVIDER_AUTH_SCHEME=Bearer

Do not print or expose API keys.

Important language/content rule:

- Discussion with me may be Greek.
- Repository content must remain English.
- Do not propose Greek filenames, comments, code, docs, issue titles, or AI logs.
- Do not suggest deployment.
- Do not use Vercel.
- Do not create `.env`.
- Do not print secrets.

Canonical repository files:

Treat these as authoritative:

- AGENTS.md
- README.md
- .env.example
- .gitignore
- docker-compose.yml
- deploy/nginx/default.conf
- docs/specs/SPECIFICATIONS.md
- docs/specs/README.md
- docs/architecture-summary.md
- docs/final-verification.md
- docs/submission-checklist.md
- docs/README.md
- docs/ai/prompt-log.md
- docs/ai/codex-runs/
- docs/ai/prompts/README.md
- docs/diagrams/final/

If anything conflicts with AGENTS.md or docs/specs/SPECIFICATIONS.md, follow AGENTS

Accepted final architecture:

1. Frontend:
   - one role-based React + TypeScript + Vite frontend;
   - located under apps/frontend;
   - browser calls only gateway-relative `/api/...` routes;
   - no browser calls to internal service ports;
   - no browser calls to external provider APIs;
   - no browser RabbitMQ access.
```

2. Backend:
   - Python FastAPI microservices;
   - each stateful service owns its data and its own PostgreSQL container;
   - no MongoDB;
   - no cross-service SQL joins;
   - no service writes to another service database.

3. Gateway:
   - Nginx reverse proxy/API gateway;
   - public frontend/API access goes through gateway;
   - provider services are internal;
   - service-owned `/internal/jobs/*` routes are blocked from gateway with 404;
   - scheduler operator APIs are exposed under `/api/scheduler/*`.

4. Runtime:
   - Docker Compose.

5. Databases:
   - PostgreSQL only;
   - separate PostgreSQL container per stateful service.

6. Messaging:
   - RabbitMQ topic exchange;
   - used only for background/event flows:
     - provider/catalog sync propagation where applicable;
     - reservation outbox;
     - analytics ingestion/aggregation;
     - billing propagation/outbox.
   - not used for:
     - login;
     - map search;
     - point details;
     - status lookup;
     - live reservation provider calls;
     - direct frontend actions.

7. Scheduler/control-plane:
   - Phase C added `scheduler-service`;
   - Phase C added `scheduler-postgres`;
   - scheduler stores only scheduler metadata:
     - scheduled jobs;
     - run history;
     - status;
     - safe summaries.
   - scheduler does not contain business logic;
   - scheduler does not read/write other service databases;
   - scheduler triggers service-owned internal HTTP job endpoints over Docker-inter
   - internal job endpoints require `X-Scheduler-Token` and `X-Job-Run-Id`;
   - internal job endpoints are blocked at the Nginx gateway and are not frontend/b

Final canonical service set after Phase C:

```
- auth-service
- provider-registry-service
- red-provider-service
- green-provider-service
- blue-provider-service
- catalog-service
- status-service
- reservation-service
- analytics-service
- billing-service
- scheduler-service

Stateful service databases:

- auth-postgres
- provider-registry-postgres
- red-provider-postgres
- green-provider-postgres
- blue-provider-postgres
- catalog-postgres
- status-postgres
- reservation-postgres
- analytics-postgres
- billing-postgres
- scheduler-postgres

Data/service ownership:

- auth-service:
  - users;
  - password hashes;
  - JWTs;
  - refresh tokens;
  - roles;
  - provider-admin scope;
  - Google OAuth hooks/config.

- provider-registry-service:
  - provider metadata;
  - provider activation status;
  - provider API configuration metadata;
  - safe config audit logs.

- red-provider-service / green-provider-service / blue-provider-service:
  - provider-local snapshots;
  - raw payloads;
  - sync runs;
  - provider-specific normalization;
  - provider status/reservation adapter calls;
  - Bearer auth toward live provider APIs.

- catalog-service:
```

```
  - aggregated normalized charging-point catalog;
  - viewport/bounds map loading;
  - cached point details;
  - import runs;
  - freshness metadata;
  - scheduler-triggered catalog import endpoints.

- status-service:
  - 5-minute status cache;
  - status check logs;
  - fallback to last known status on timeout/error;
  - status normalization to `unknown` for unexpected provider statuses.

- reservation-service:
  - reservation truth;
  - synchronous provider reservation calls through provider services;
  - successful/failed/unknown attempts;
  - reservation event outbox;
  - scheduler-triggered outbox publishing.

- analytics-service:
  - idempotent raw usage_events;
  - aggregate/cache tables;
  - provider/operator analytics;
  - CSV export;
  - no direct SQL access to billing/reservation/catalog DBs;
  - scheduler-triggered event consumption and aggregate refresh.

- billing-service:
  - pricing rules;
  - charges;
  - invoices;
  - invoice lines;
  - mock full-invoice payments;
  - billing event ingestion/outbox;
  - financial source of truth;
  - scheduler-triggered event drain, outbox publish, and invoice-preparation summar

- scheduler-service:
  - scheduled job definitions;
  - scheduler run metadata;
  - operator-only scheduler APIs;
  - internal service call status;
  - safe response summaries only;
  - no business data ownership.

- packages/events:
  - shared event envelope;
  - routing-key constants;
  - RabbitMQ topology declarations;
  - publisher/consumer helpers;
  - idempotency helpers.
```

Implementation history:

The project was implemented through staged Codex prompts and GitHub issues.

Original implementation project:

- Issue #1 / Prompt 001 - repository foundation
- Issue #2 / Prompt 002 - Docker Compose and Nginx gateway
- Issue #3 / Prompt 003 - auth service
- Issue #4 / Prompt 004 - provider registry service
- Issue #5 / Prompt 005 - red/green/blue provider services
- Issue #6 / Prompt 006 - catalog service
- Issue #7 / Prompt 007 - status service
- Issue #8 / Prompt 008 - reservation service
- Issue #9 / Prompt 009 - RabbitMQ event infrastructure
- Issue #10 / Prompt 010 - analytics service
- Issue #11 / Prompt 011 - billing service
- Issue #12 / Prompt 012 - EV frontend flows
- Issue #13 / Prompt 013 - provider/operator dashboards
- Issue #14 / Prompt 014 - final docs and UML

Finalization/hardening project:

- Issue #15 / Prompt 015 - Phase A finalization project creation and implementatior
- Issue #16 / Prompt 016 - Phase B live environment and real provider verification
- Issue #18 / Prompt 017/018 - Phase C scheduler/control-plane design and implement
- Issue #17 / Prompt 019/020 - Phase D comprehensive test expansion and final proje

Important final status from previous work:

Phase A:
- closed old implementation GitHub Project;
- created finalization/hardening GitHub Project;
- created Phase A-D issues;
- reran repository audit;
- validated/rendered PlantUML diagrams;
- Phase A completed.

Phase B:
- `.env` remained untracked;
- live provider credentials were verified;
- redPlug list/detail passed;
- greenPlug list/detail passed;
- bluePlug list/status passed;
- provider services were fixed to use Bearer auth;
- Google OAuth non-interactive configuration passed;
- authenticated EV/provider/operator smokes passed;
- live reservation mutation smoke intentionally skipped;
- Phase B issue #16 closed/done.

Phase C:

- added scheduler-service and scheduler-postgres;
- added scheduler metadata persistence;
- added scheduler jobs and run history;
- added operator-only scheduler APIs;
- added internal job endpoints to catalog/analytics/billing/reservation;
- added operator scheduler UI support;
- added Nginx `/api/scheduler/*`;
- blocked gateway access to service-owned `/internal/jobs/*` routes with 404;
- verified scheduler manual run;
- Phase C issue #18 closed/done.

Phase D:
- expanded tests across backend services, frontend, shared events, and root gateway
- added root-level `tests/` directory;
- verified 184 tests total after the comprehensive continuation;
- closed Phase D issue #17;
- closed finalization GitHub Project #2.

Known final non-blocking limitations / warnings:

- Full interactive Google OAuth browser callback was not run; non-interactive OAutl
- Live reservation mutation smoke was intentionally skipped by safety policy.
- Browser visual QA/Figma QA was not performed because no usable Browser/Figma sour
- During final Phase D full-stack startup, first `docker compose up -d --build` hac
- These are not considered blockers.

Your task in this new ChatGPT conversation:

Use the GitHub connector to inspect the actual repository deeply. Do not rely only

Also use the project sources and saved memory/context available to you for the NTUA

Inspect at least:

Root/governance:
- AGENTS.md
- README.md
- .gitignore
- .env.example
- docker-compose.yml
- deploy/nginx/default.conf

Specs/final docs:
- docs/specs/SPECIFICATIONS.md
- docs/specs/README.md
- docs/architecture-summary.md
- docs/final-verification.md
- docs/submission-checklist.md
- docs/README.md

AI logs:
- docs/ai/prompt-log.md

```
- docs/ai/codex-runs/014-final-docs-uml.md
- docs/ai/codex-runs/015-finalization-project-audit.md
- docs/ai/codex-runs/016-live-env-verification.md
- docs/ai/codex-runs/017-scheduler-design.md
- docs/ai/codex-runs/018-scheduler-control-plane-implementation.md
- docs/ai/codex-runs/019-phase-d-test-expansion.md
- docs/ai/codex-runs/020-comprehensive-test-expansion-and-project-closure.md
- docs/ai/prompts/

Frontend:
- apps/frontend/README.md
- apps/frontend/package.json
- apps/frontend/src/App.tsx
- apps/frontend/src/features/
- apps/frontend/src/lib/
- apps/frontend/src/test/

Shared event package:
- packages/events/README.md
- packages/events/saasplug_events/
- packages/events/tests/

Services:
- services/README.md
- services/auth-service/
- services/provider-registry-service/
- services/red-provider-service/
- services/green-provider-service/
- services/blue-provider-service/
- services/catalog-service/
- services/status-service/
- services/reservation-service/
- services/analytics-service/
- services/billing-service/
- services/scheduler-service/

Root tests:
- tests/README.md
- tests/test_gateway_contract.py

Diagrams:
- docs/diagrams/final/README.md
- all `.puml` files under docs/diagrams/final/

Assignment/provider sources:
- docs/assignment/
- docs/assignment/providers/yaml/redPlug.yaml
- docs/assignment/providers/yaml/greenPlug.yaml
- docs/assignment/providers/yaml/bluePlug.yaml
- docs/assignment/providers/postman_collections/redPlug.postman_collection.json
- docs/assignment/providers/postman_collections/greenPlug.postman_collection.json
- docs/assignment/providers/postman_collections/bluePlug.postman_collection.json
```

```
GitHub:
- inspect issues #1 through #18 if possible;
- especially issue #17 and #18;
- confirm closed/completed status where visible;
- confirm no fallback Markdown project-management artifacts such as `docs/project-m
- confirm Project #2 closure if visible; if not visible through connector, say that

Questions to answer from actual repo evidence:

1. Is the full implementation complete enough for final NTUA course submission?
2. Does the implementation match the accepted architecture after the Phase C schedu
3. Are all services present and correctly separated by responsibility?
4. Does the frontend call only gateway `/api/...` paths?
5. Are provider services kept internal and not exposed directly to the browser?
6. Is RabbitMQ used only for background/event flows, not live frontend actions?
7. Is the one-PostgreSQL-container-per-stateful-service rule preserved, now includi
8. Are service-owned `/internal/jobs/*` routes blocked at the gateway?
9. Is scheduler-service correctly implemented as control-plane, not business orches
10. Are live provider credentials/auth assumptions correctly reflected in the code/
11. Are final docs and PlantUML source diagrams present and coherent?
12. Are AI logs complete and consistent from 000/00B through 020?
13. Are tests broad enough after Phase D?
14. Are there any blockers before final submission?
15. Are there non-blocking warnings that should remain documented?
16. Is the repository ready for final course submission?

Then produce your final answer in Greek with this exact structure:

## Final repository review result

Concise but evidence-based verdict.

## What is confirmed

Use a table with:
- Area
- Status
- Evidence / notes

## Warnings / non-blocking limitations

Mention only real issues grounded in repo evidence.

## Blockers

Say “No blockers” only if repository evidence supports it.

## Final readiness

State whether it is ready for final course submission.
```

```
## Final Codex full verification prompt

Give me one exact copy-paste prompt for Codex to run a final verification-only pass

The final Codex prompt must:
- not implement new features;
- not change architecture;
- not change backend/frontend business logic;
- not add deployment/Vercel;
- not create `.env`;
- not print secrets;
- not expose provider API keys, JWT secrets, Google secrets, RabbitMQ passwords, da
- read canonical docs and AI logs;
- verify `.env.example` completeness;
- verify `.env` is untracked without printing it;
- run frontend tests/build;
- run all backend service tests;
- run scheduler-service tests;
- run shared event tests;
- run root gateway contract tests;
- run Docker Compose config/startup/health;
- verify gateway/public health routes;
- verify authenticated EV/provider/operator smoke checks;
- verify `/api/scheduler/health`;
- verify scheduler operator API smoke;
- verify service-owned `/internal/jobs/*` gateway paths return 404;
- verify scheduler manual run smoke for a safe non-mutating job such as `billing.pr
- verify provider live read-only checks only if real credentials/VPN are available;
- skip live reservation mutations unless explicitly enabled by `RUN_LIVE_RESERVATI(
- verify frontend source has no forbidden direct service/provider/RabbitMQ calls;
- verify final docs/diagrams are present;
- run PlantUML validation if available;
- update or create a final verification run report under docs/ai/codex-runs/;
- update docs/ai/prompt-log.md only if it creates a new verification run entry;
- update docs/final-verification.md and docs/submission-checklist.md only if verifi
- clearly report unavailable tools instead of claiming success;
- not create fallback project-management Markdown artifacts;
- not close/reopen issues or projects unless explicitly instructed.

Important:
The final Codex prompt must be verification-only, not implementation.
Do not give generic advice. Ground all conclusions in the actual repository.
```

This is the exact objective for the new conversation: **full independent repository review + one final Codex verification-only prompt**.